

Pipelining

João Canas Ferreira / António José Araújo

Março de 2025

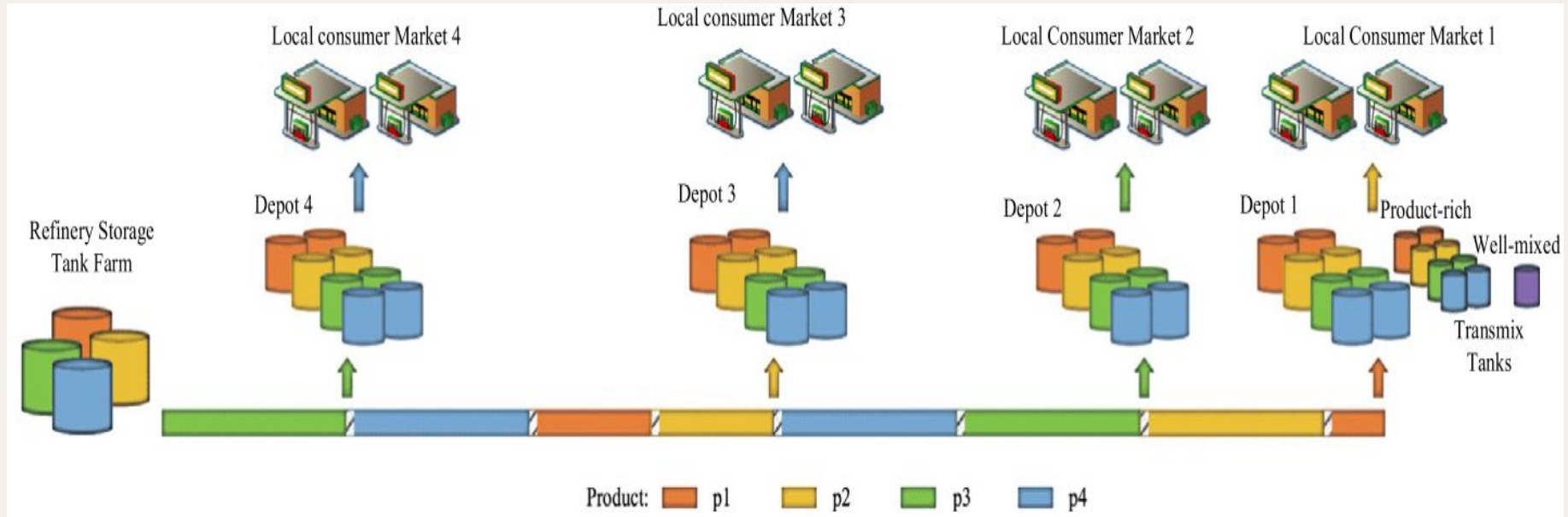
Execução concorrente de instruções

Como melhorar o desempenho do CPU?

- Executar mais que uma instrução em simultâneo!
- O mesmo problema ocorre nos processos de fabrico.
 - Por exemplo, como fabricar mais que um automóvel sem duplicar a fábrica?
- Solução: Linha de montagem móvel

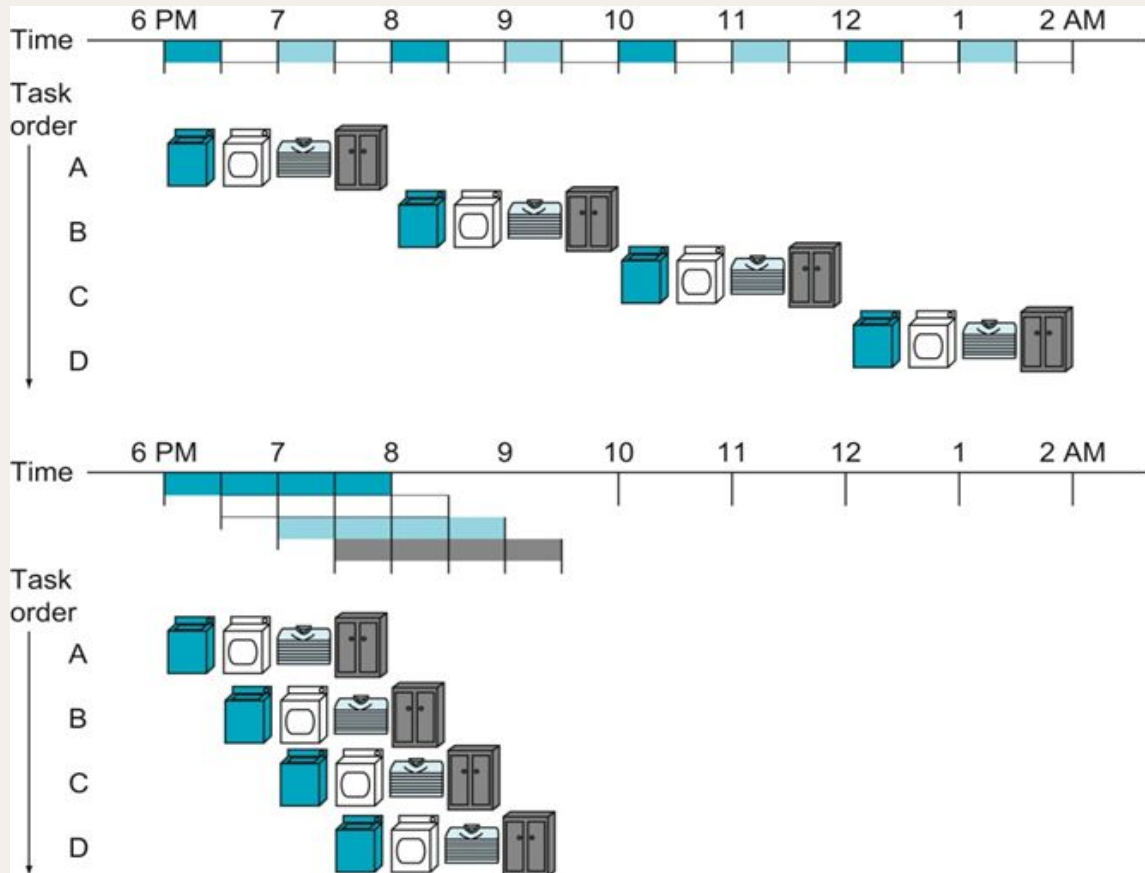


Outra analogia: oleodutos (*pipelines*)



Li Yu, Min Chen, Qiang Xu,
Simultaneous scheduling of multi-product pipeline distribution and depot inventory management
for petroleum refineries,
Chemical Engineering Science, vol. 220, 2020, doi: 10.1016/j.ces.2020.115618.

Sobreposição da execução



- Quatro tarefas
- Cada tarefa: 0.5 h

Aumento de rapidez

- Uma carga:
 $2/2 = 1$
- Quatro cargas:
 $8/3.5 = 2.3$

$$\frac{2 \times N}{0.5 \times N + 1.5}$$

$$\lim_{N \rightarrow \infty} \frac{2 \times N}{0.5 \times N + 1.5} = 4$$

Pipeline de referência para RISC-V

Cinco andares (ou estágios), um passo por andar

1. IF: (*Instruction fetch*) Obter instrução de memória (I-Cache)
2. ID: (*Instruction decode*) Descodificação e leitura de registos
3. EX: (*Execute*) Executar uma operação ou calcular um endereço de memória
4. MEM: Acesso a memória (D-Cache)
5. WB: (*Write back*) Guardar resultado num registo

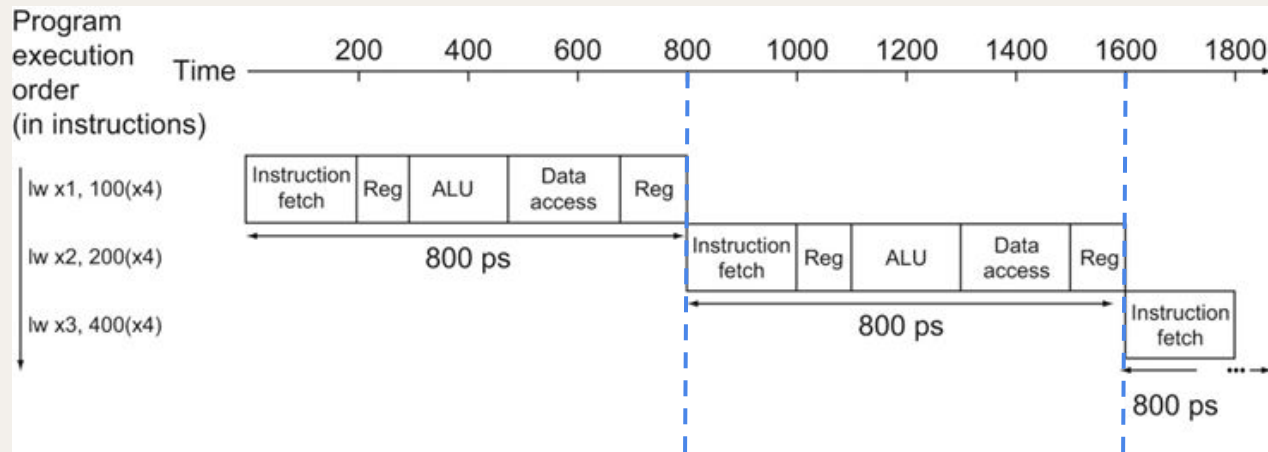
Desempenho com *pipeline* (1/3)

Assumir que tempo necessário para cada andar é:

- 100ps para leitura ou escrita em registros
- 200ps para os outros andares

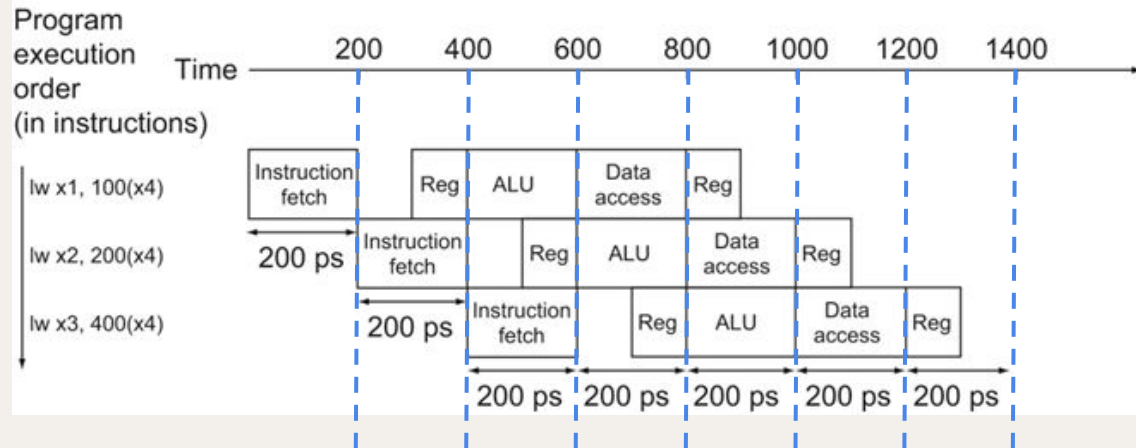
Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load word (lw)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (sw)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format (add, sub, and, or)	200 ps	100 ps	200 ps		100 ps	600 ps
Branch (beq)	200 ps	100 ps	200 ps			500 ps

Desempenho com *pipeline* (2/3)



1 ciclo por instrução
($T_c = 800$ ps)

Período definido pela
instrução mais demorada



Pipeline
($T_c = 200$ ps)

Período definido pela
etapa mais demorada

Desempenho com *pipeline* (3/3)

- Situação ideal: todos os andares têm o mesmo tempo de execução (*pipeline em equilíbrio*).
- Em condições ideais:

$$\text{Tempo entre instruções}_{\text{pipeline}} = \frac{\text{Tempo entre instruções}_{\text{sem pipeline}}}{\text{Número de andares}}$$

- O exemplo tem poucas instruções
- Com mais 1 milhão de instruções

$$\text{Speedup} = \frac{2400 \text{ ps}}{1400 \text{ ps}} = 1.71$$

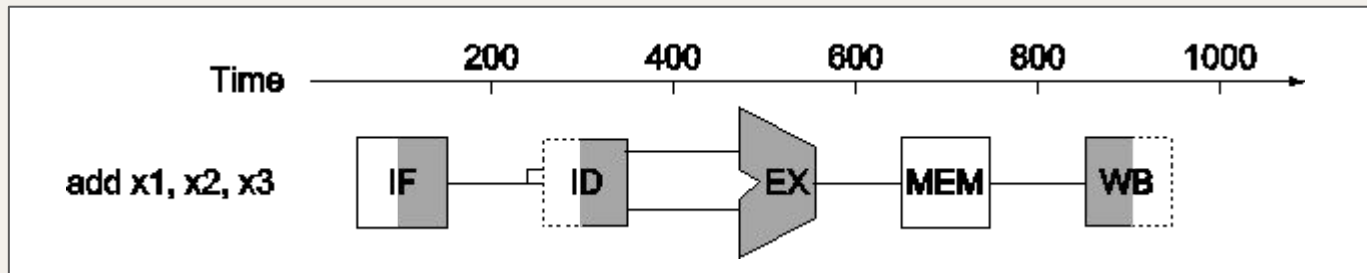
$$\text{Speedup} = \frac{10^6 \times 800 + 2400}{10^6 \times 200 + 1400} = \frac{800002400}{200001400} \approx 3.9999$$

- A melhoria de desempenho é causada pelo aumento do débito (nº de instruções executadas por unidade de tempo), não pela redução do tempo de execução das instruções individuais.
- De facto, **nenhuma** instrução fica individualmente mais rápida.

Influência do conjunto de instruções

- As características do conjunto de instruções podem facilitar a construção de *pipelines*.
- No caso do conjunto de instruções RISC-V:
 - a. Instruções têm todas o mesmo tamanho: facilita a obtenção de instruções (IF) e a sua decodificação (ID).
 - b. Vários formatos diferentes, mas com a especificação dos registros (usados para leitura de operandos) nas mesmas posições: permite obter os operandos ao mesmo tempo que se procede à decodificação (ID).
 - Não é preciso saber qual é a instrução para consultar os registros
 - c. Acessos a memória apenas em instruções do tipo *load/store*: o andar de execução pode ser usado para efetuar o cálculo do endereço efetivo (EX).

Representação gráfica da *pipeline* de referência



- Existem duas memórias:
 - a. memória de instruções: usada na etapa IF (I-Cache)
 - b. memória de dados: usada na etapa MEM (D-Cache)
- A cinzento: elementos usados no processamento da instrução.
- Para os elementos de memória (ID e WB):
 - a. cinzento do lado esquerdo: escrita
 - b. cinzento do lado direito: leitura
- Banco de registos: leitura no fim do ciclo; escrita no início do ciclo.

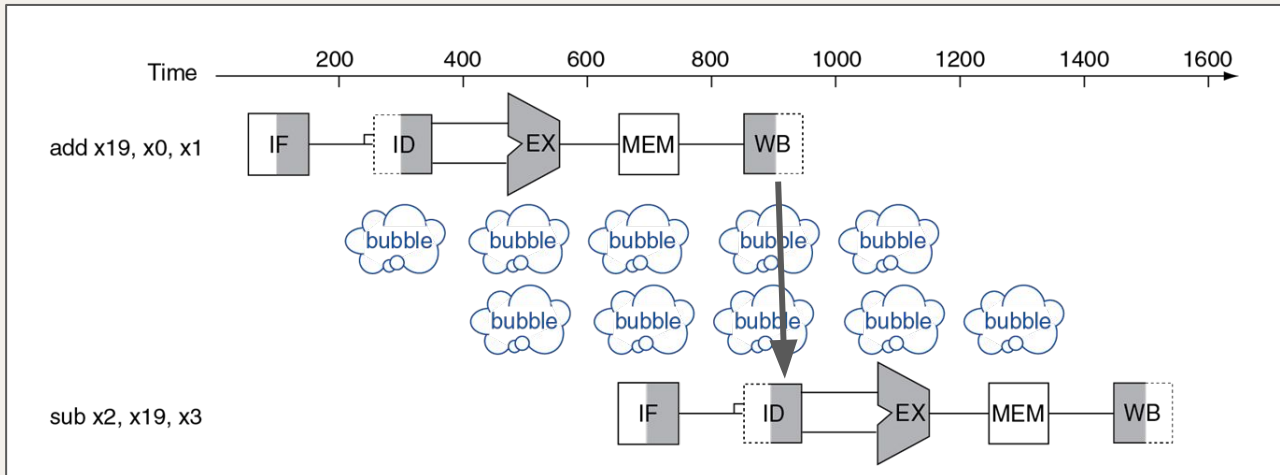
Dependências entre instruções

Obstáculos ao encadeamento de instruções

- Existem situações em que a instrução seguinte não pode começar a execução no ciclo seguinte: a nova instrução estaria em conflito com as instruções ainda em execução. (Em inglês: *hazard*; não é uma tradução literal).
- Conflitos estruturais
 - O suporte físico não permite a execução concorrente (falta de recursos).
 - Exemplo: memória (se não existirem memórias caches separadas).
- Conflitos de dados
 - Os dados necessários não estão disponíveis, porque são produzidos por uma instrução que ainda não terminou.
- Conflitos de controlo
 - A instrução a executar é (ainda) desconhecida.
Exemplo: Não é conhecido o resultado do teste incluído no salto condicional.

Uma solução: protelamento

- Todos os conflitos podem ser resolvidos por protelamento (espera):
 - A instrução cuja execução devia ser iniciada “espera” pelo fim da execução da instrução com que está em conflito.
- Conflitos não tratados levam a cálculos com valores errados (“antigos” valores dos registos, por exemplo).

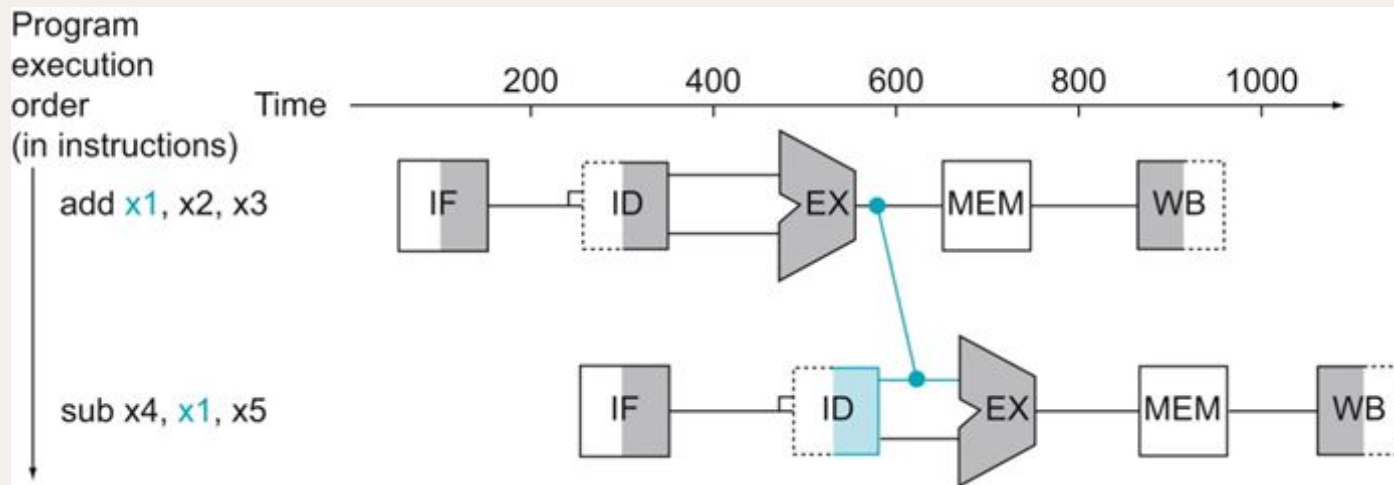


```
add x19, x0, x1
sub x2, x19, x3
```

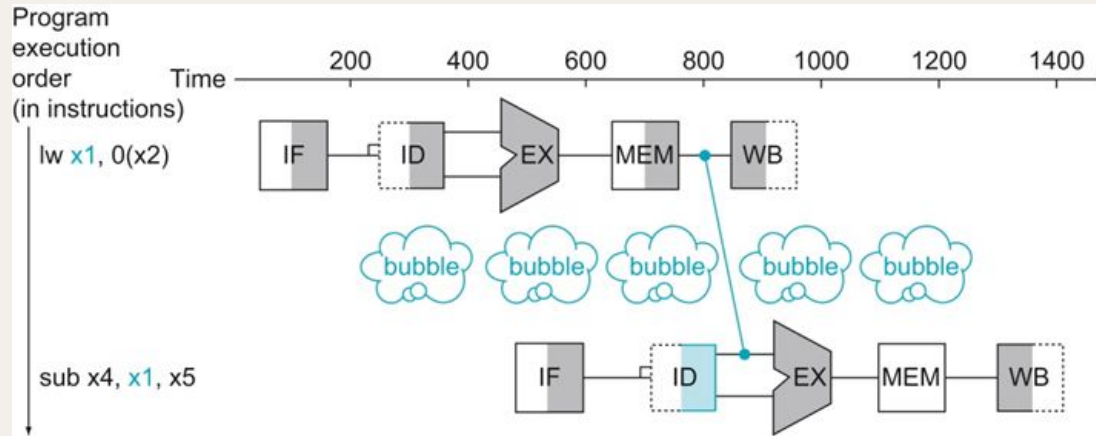
Não é, geralmente, uma boa solução, porque reduz o desempenho!

Solução (parcial): atalhos

- Alguns conflitos de dados podem ser resolvidos por **atalhos**. (*forwarding, bypassing*)
- Atalho: percurso alternativo interno entre os andares
(de andares mais próximos do fim para andares mais próximos do princípio)
- Condição para utilização de atalhos: o resultado (valor) requerido já deve estar disponível algures no processador (caminho de dados).



Protelamento mesmo na presença de atalhos

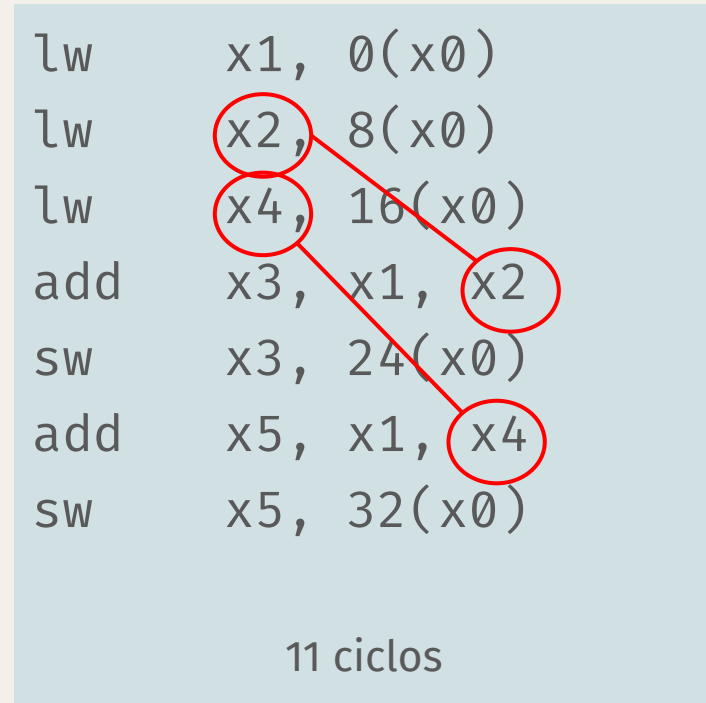
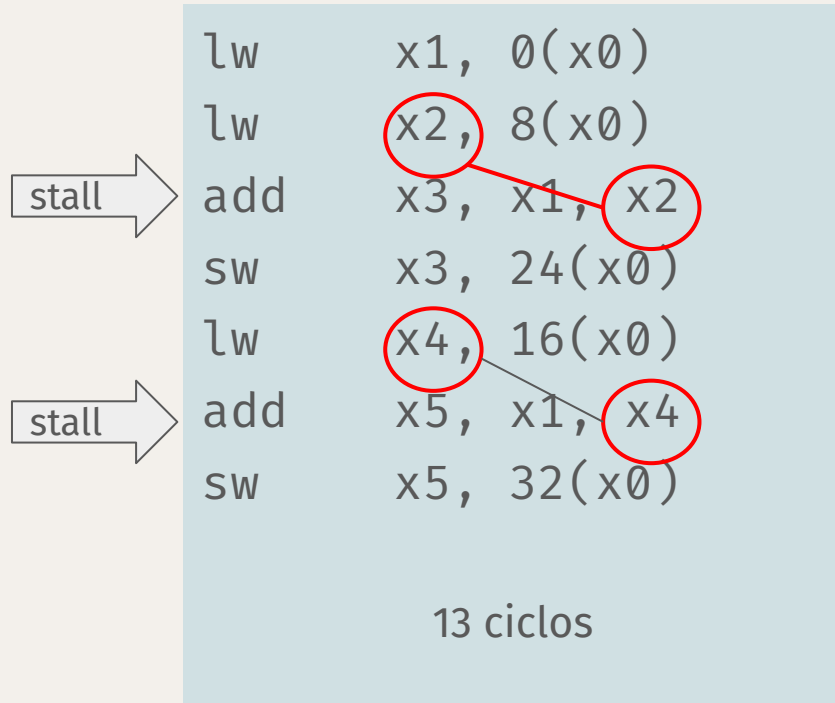


- Atalhos não resolvem todos os conflitos de dados.
 - Por exemplo, pode ser necessário esperar pelos dados fornecidos pela memória (*load-use data hazard*).
 - Sem atalhos, seria necessário protelar a instrução “sub” mais um ciclo.

Protelamentos evitados pelo compilador

$a = b + e;$ $c = b + f;$

Protelamento: “stall”

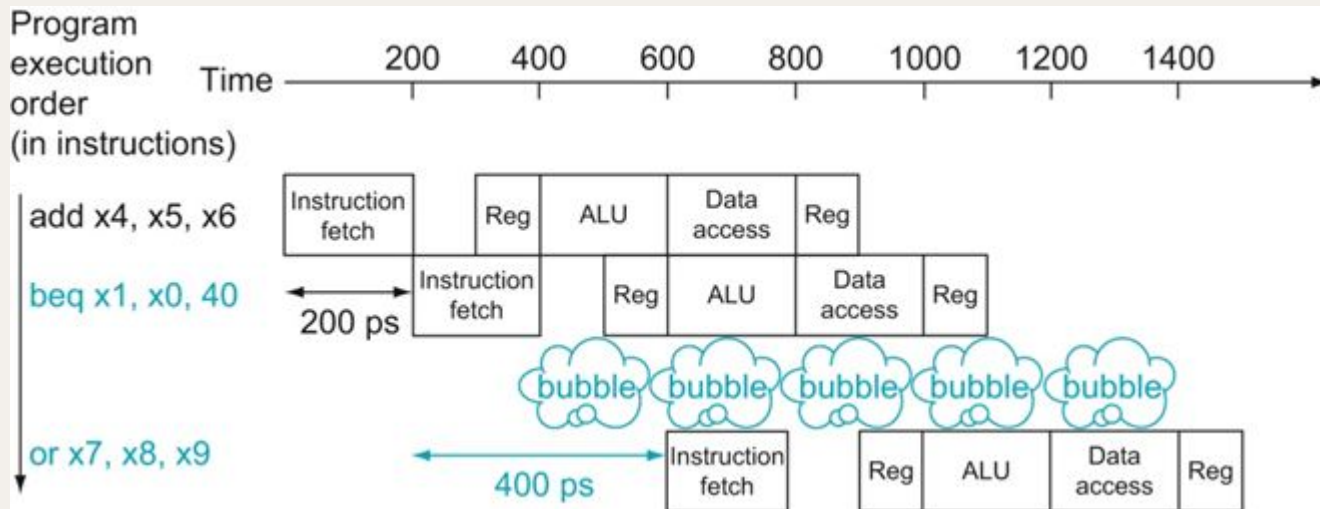


Conflitos de controlo

- Saltos condicionais (*branches*) determinam o fluxo de controlo
 - A busca da próxima instrução depende do resultado do teste (salto).
 - CPU nem sempre pode buscar a instrução correta:
ainda está no estágio de ID da instrução de salto.
- No *pipeline* RISC-V:
 - Cenário 1: a avaliação da condição de salto e o cálculo do endereço de destino do salto estão prontos na etapa 4 (MEM).
 - **Este é o cenário usado por omissão.**
 - Cenário 2: É vantajoso avaliar a condição de salto e calcular o destino no início do *pipeline*. Para isso é necessário ter *hardware adicional* no andar ID.

Conflito de controlo: resolução parcial por protelamento

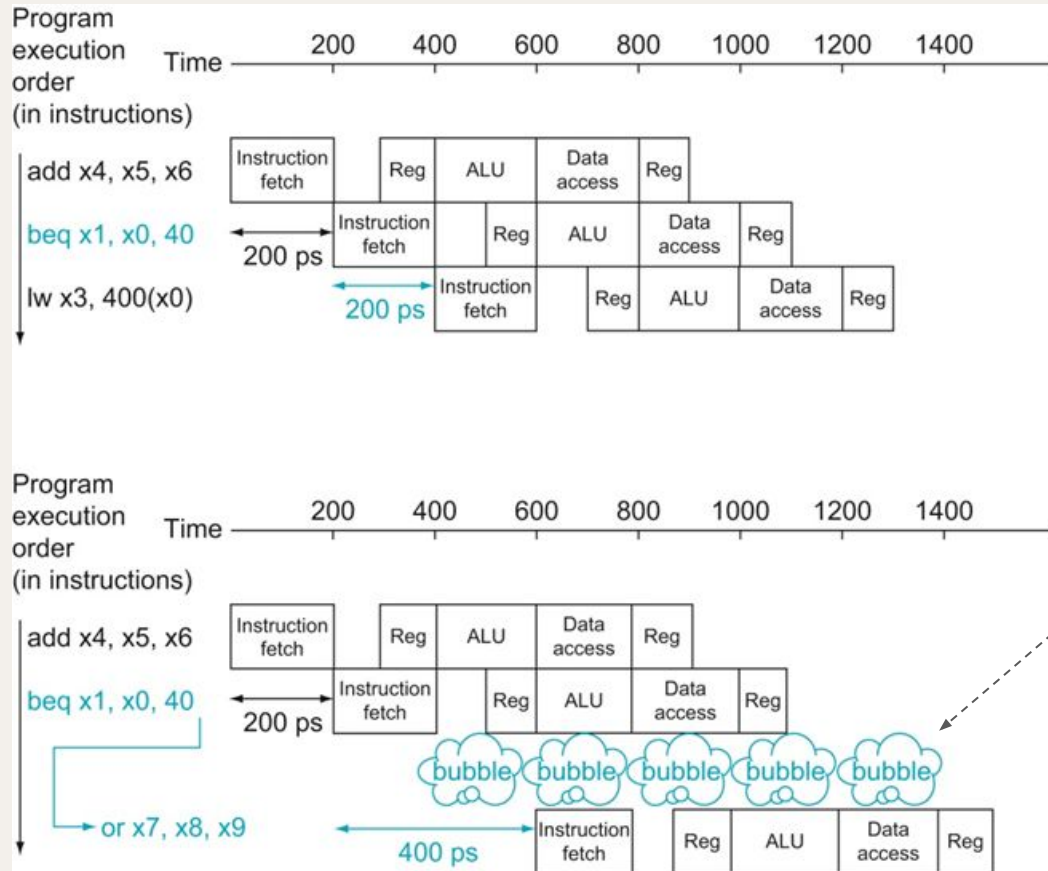
- Assumir que o caminho de dados **é alterado para determinar o resultado da comparação e calcular o destino do salto na etapa 2 (ID)!**
- Ainda assim: instrução de salto condicional requer um ciclo de protelamento.
- Em média, surge um salto condicional em cada 7-10 instruções! (SPEC)



Predição de saltos (*branch prediction*)

- Para reduzir o impacto dos saltos condicionais:
 - CPU pode tentar prever o resultado da comparação e escolher a próxima instrução com base na predição.
- Esquema simples de predição:
 - Assumir sempre que o salto **não é tomado** (a condição é falsa).
 - Caso a previsão se mostre errada, é preciso:
 - i. anular a instrução em execução;
 - ii. repetir andar IF com o endereço da instrução correta.
 - **Cenário a considerar por omissão quando existir predição.**
- Esquema ligeiramente mais sofisticado pode prever alguns saltos como tomados (saltos para trás) e outros como não tomados (saltos para a frente).
- Soluções mais sofisticadas devem ser adaptativas (dinâmicas), i.e., ter em consideração o historial dos saltos condicionais.

Aplicação de predição de saltos

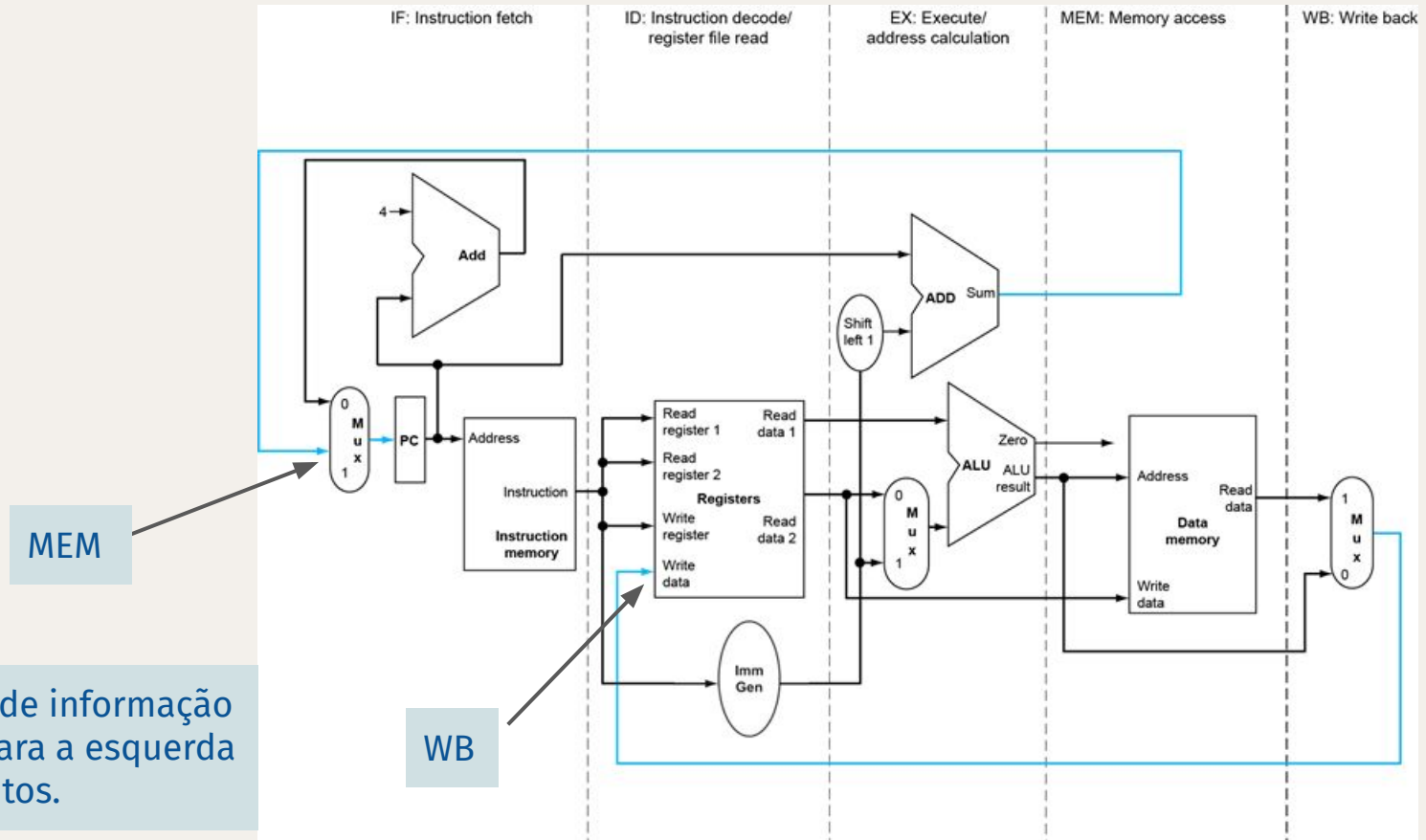


Anular a instrução porque o salto deve ser tomado.

Implementação I

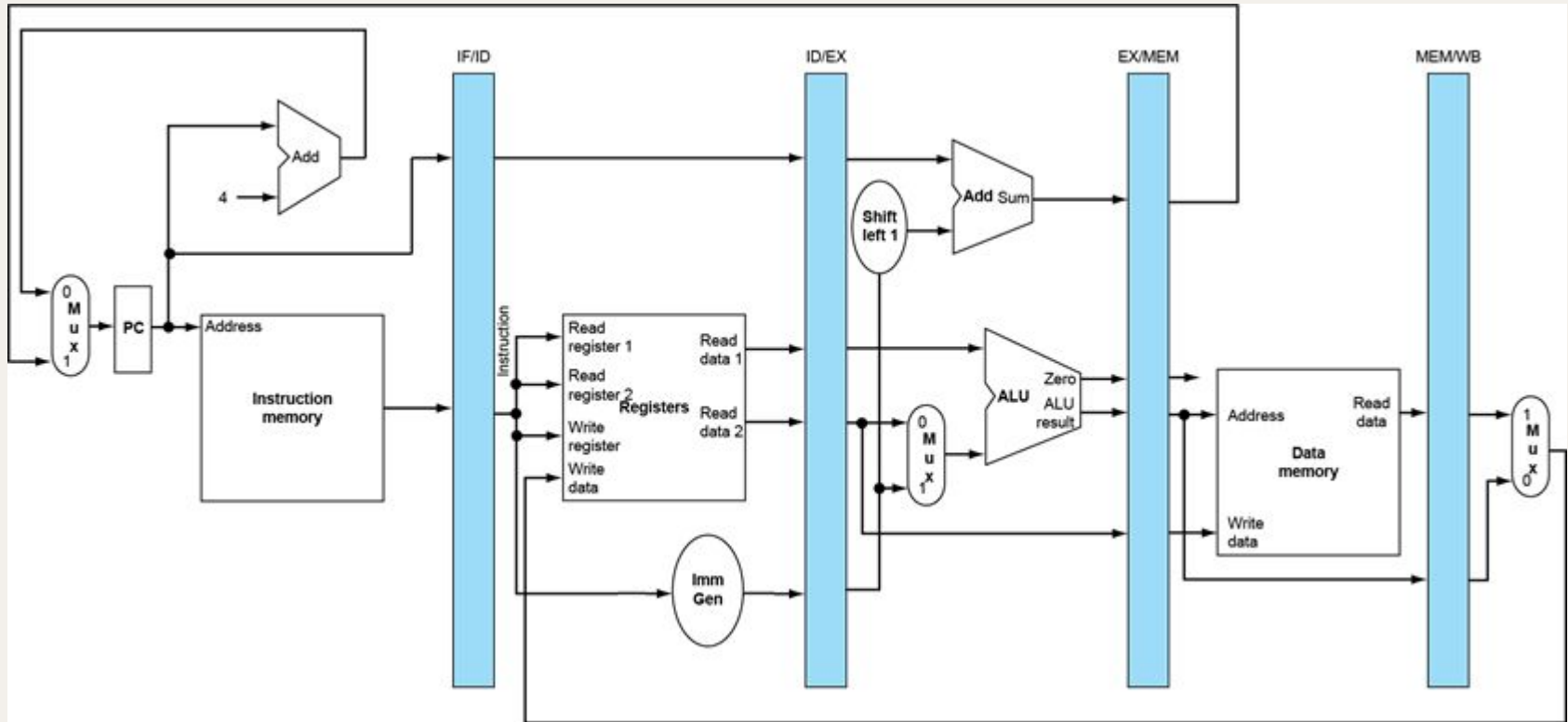
Sem tratamento de conflitos

Organização de *pipeline* para RISC-V



Registos entre andares de *pipeline*

Registos são necessários para guardar a informação produzida no ciclo anterior

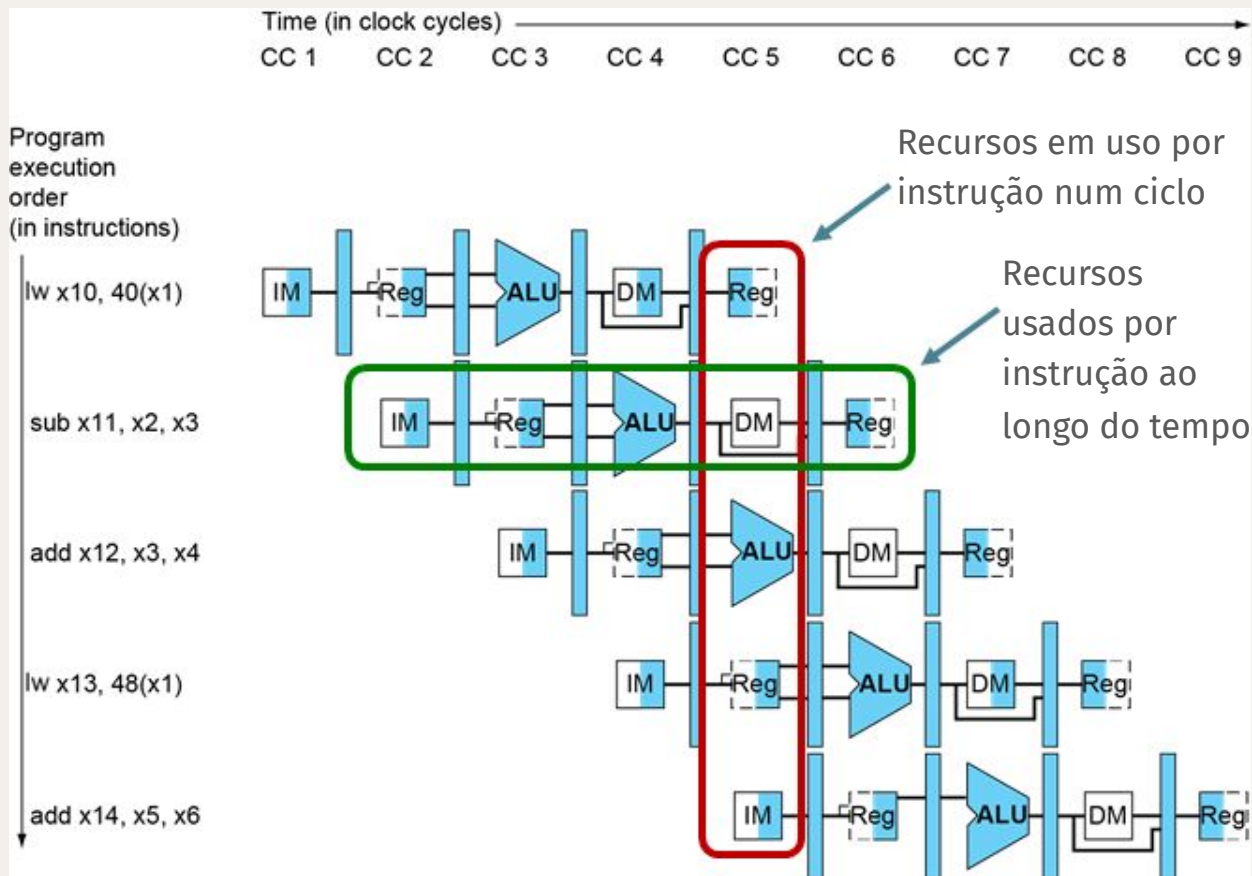


Representação de funcionamento de *pipeline*

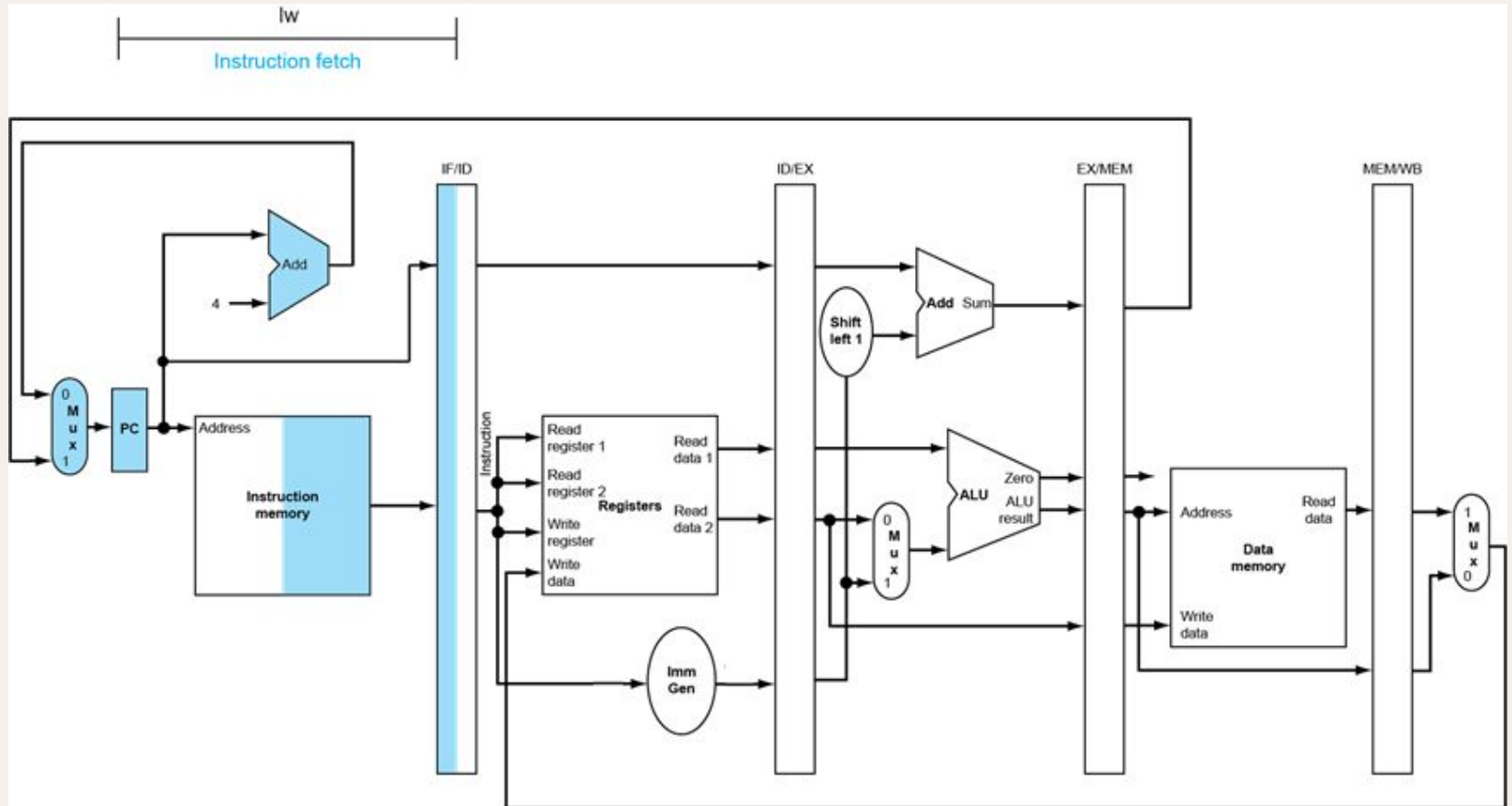
- A representação da operação da *pipeline* pode ser feita de duas perspetivas.
- Estado de *pipeline* num dado ciclo de relógio:
 - Análise do CPU num momento específico (perspetiva síncrona).
 - Retrato (*snapshot*) do estado do CPU num dado período de relógio.
 - Mostra bem a utilização dos recursos (ALU, registos, etc.).
- Evolução do tratamento de uma instrução específica ao longo do tempo:
 - Análise (parcial) do comportamento do CPU ao longo do tempo (perspetiva diacrónica).
 - Coloca o ênfase na evolução temporal.

Vamos analisar a evolução das instruções *load* e *store* →

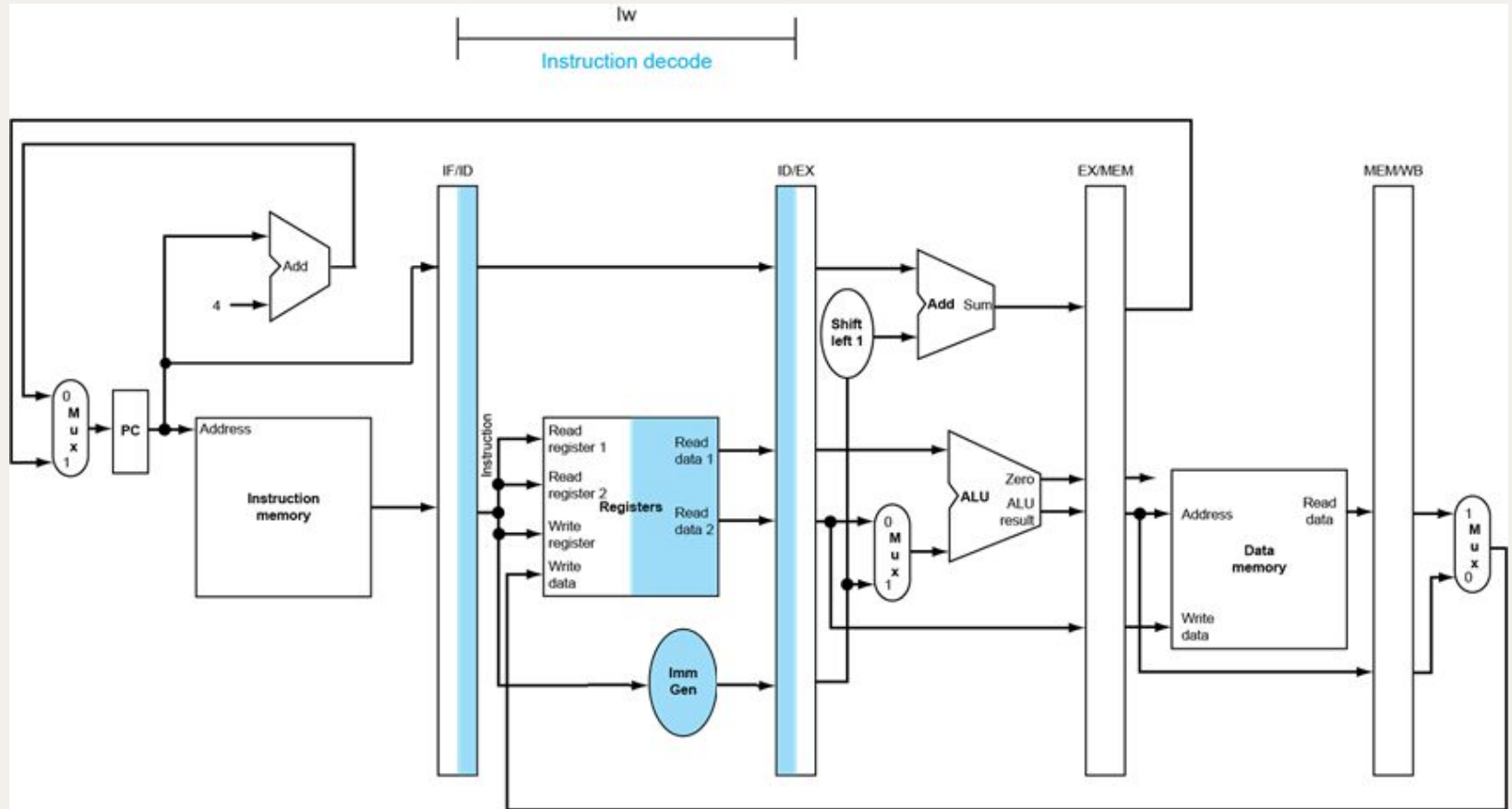
Representação de funcionamento de *pipeline*: exemplo



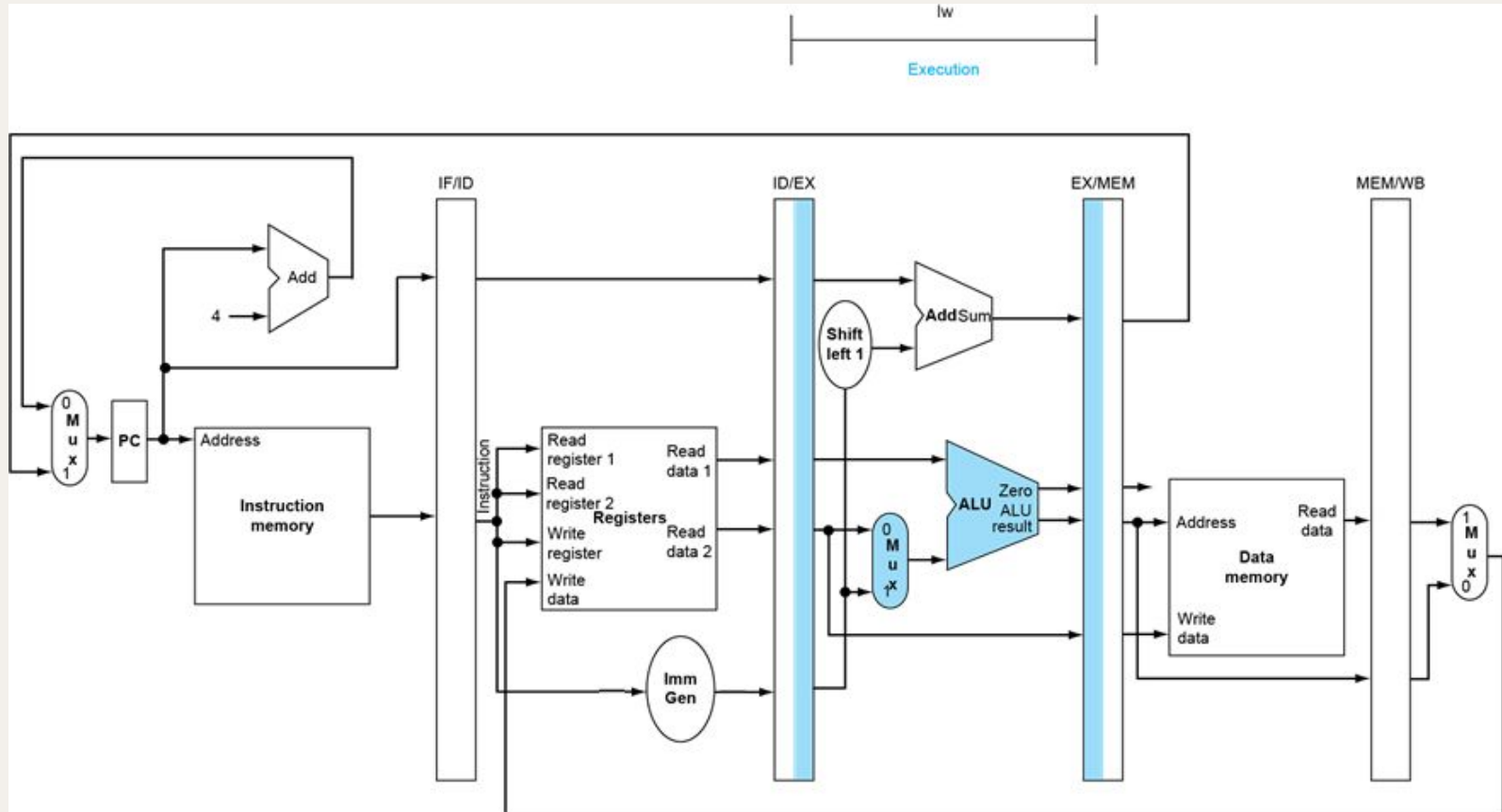
Andar IF para todas as instruções



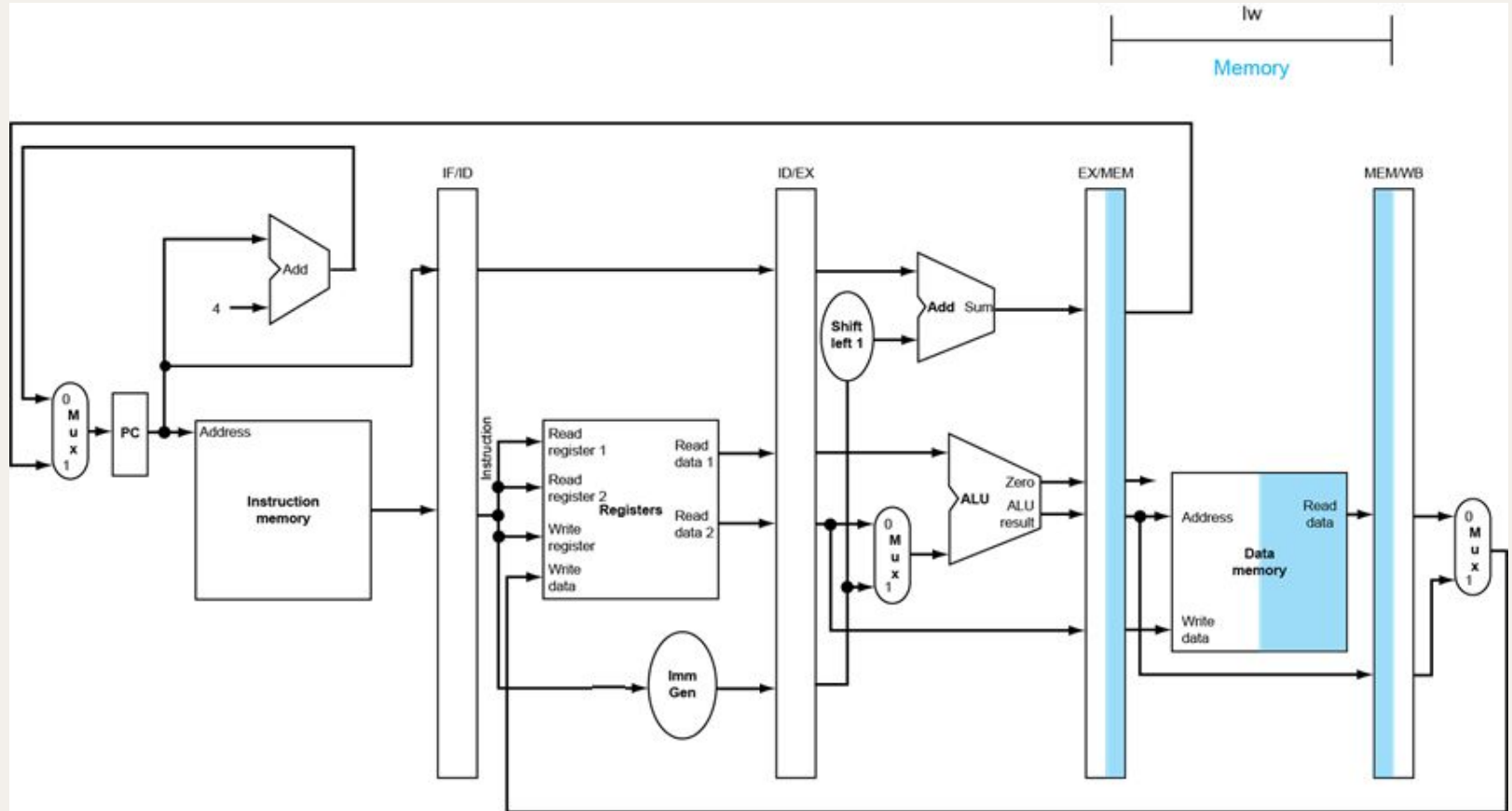
Andar ID para todas as instruções



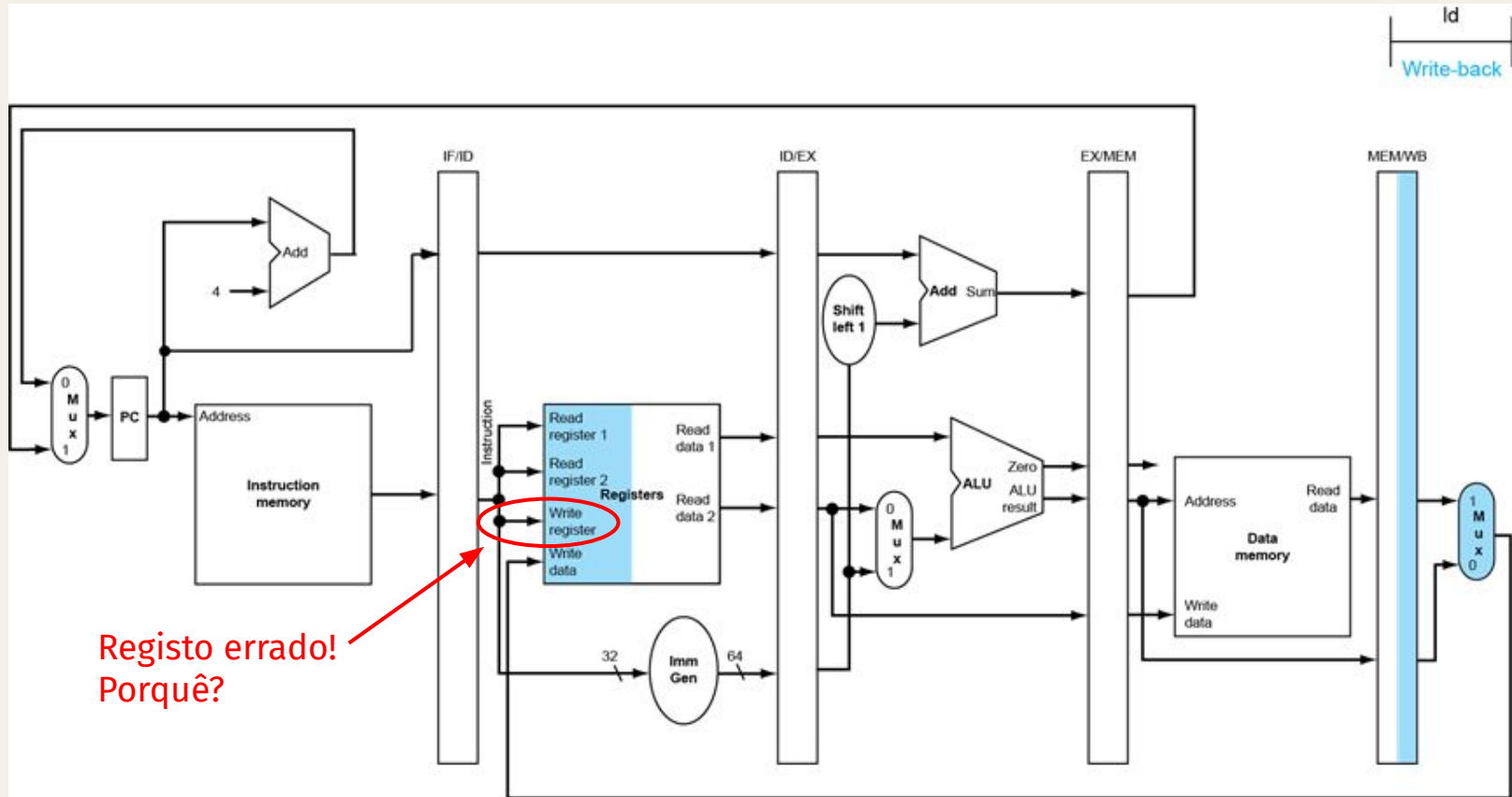
Andar EX da instrução *load*



Andar MEM para instrução *load*

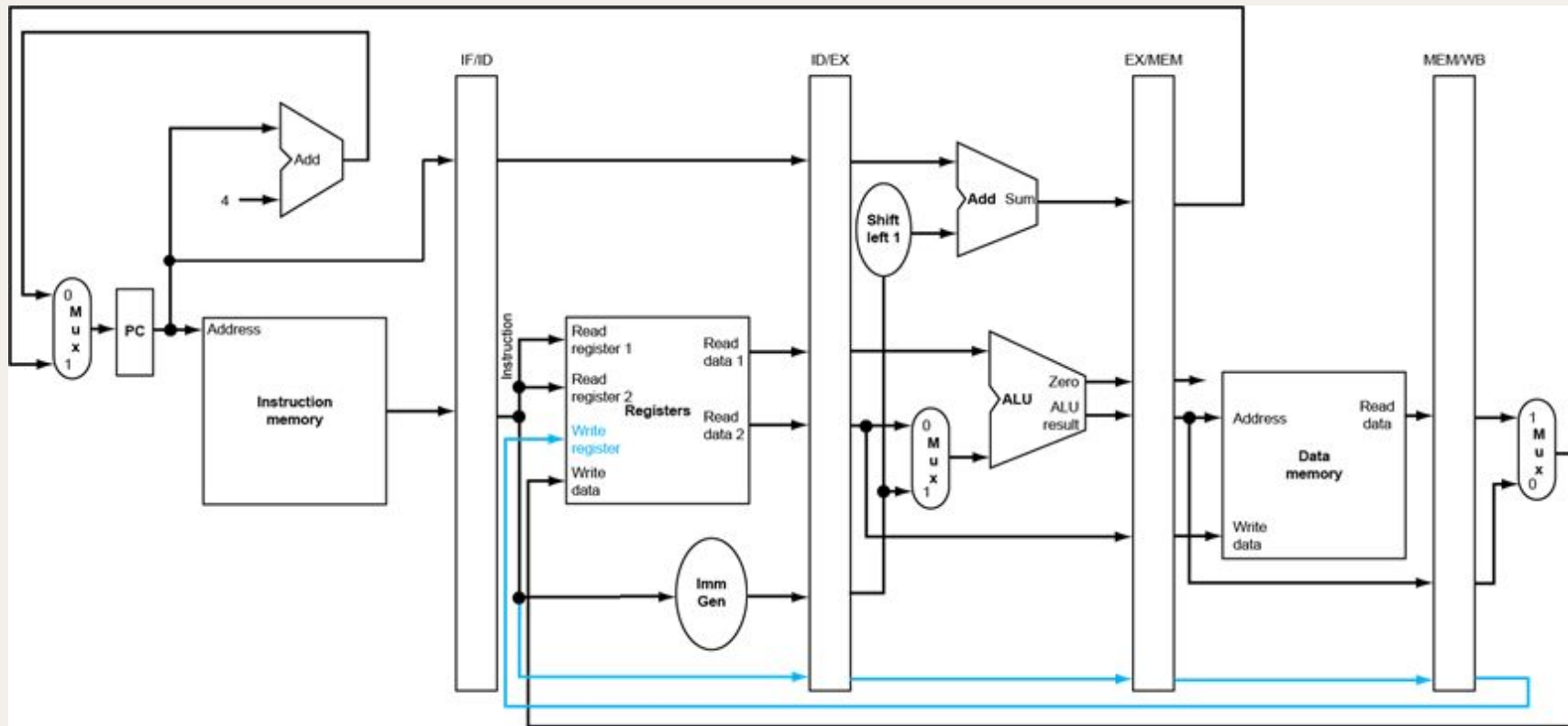


Andar WB para instrução *load* (com erro!)

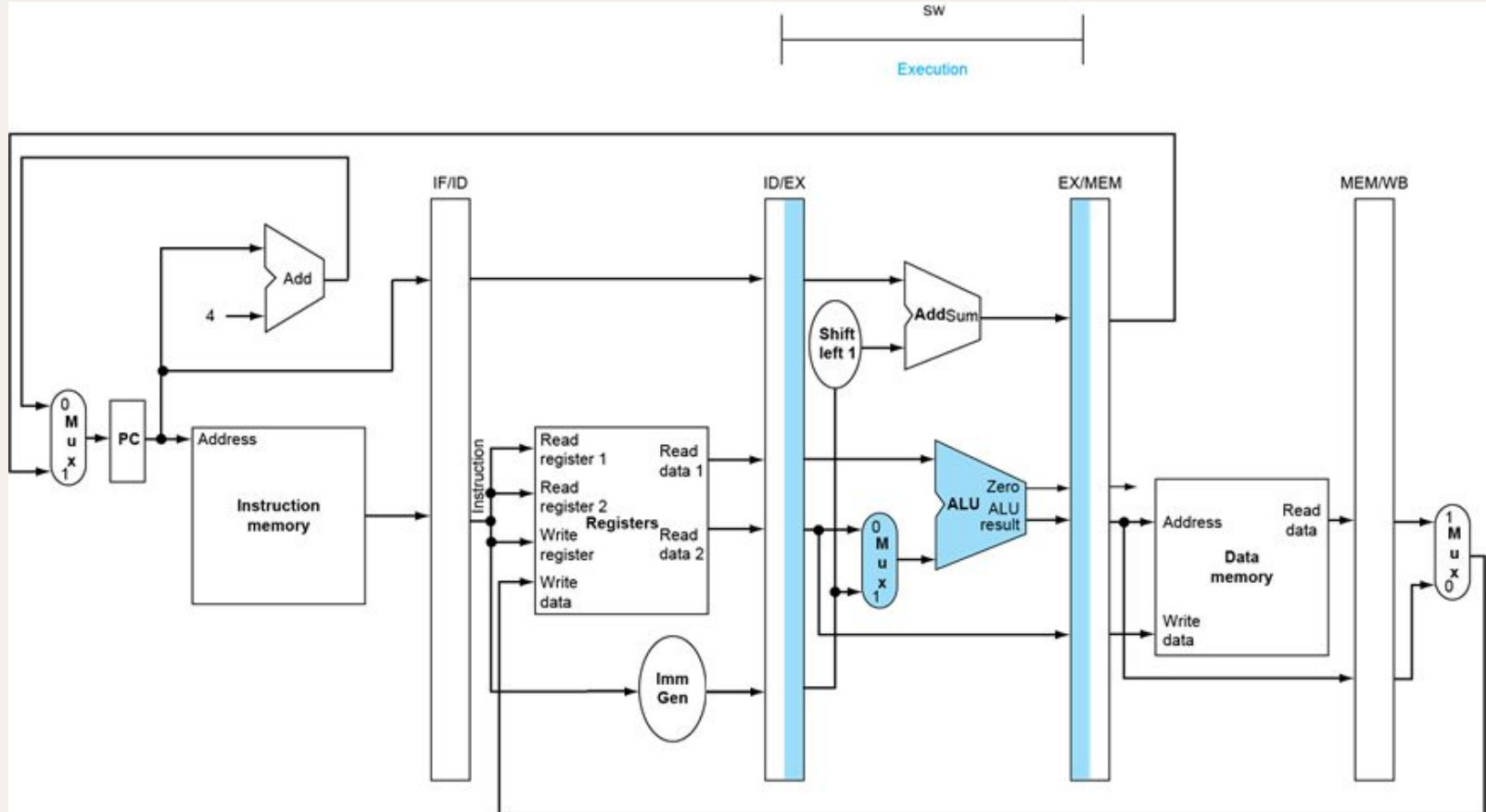


Registo errado!
Porquê?

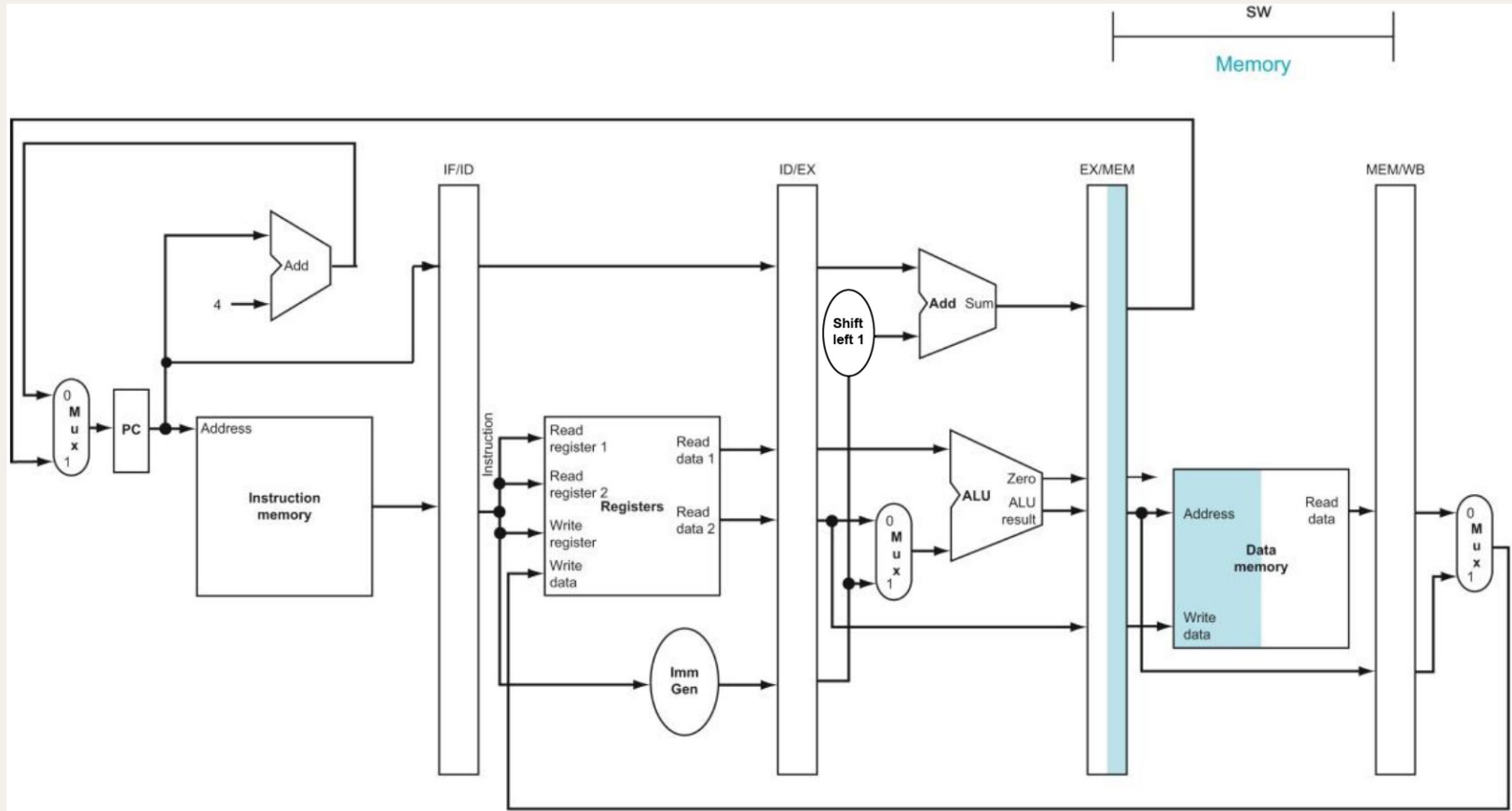
Andar WB para instrução *load* (correto!)



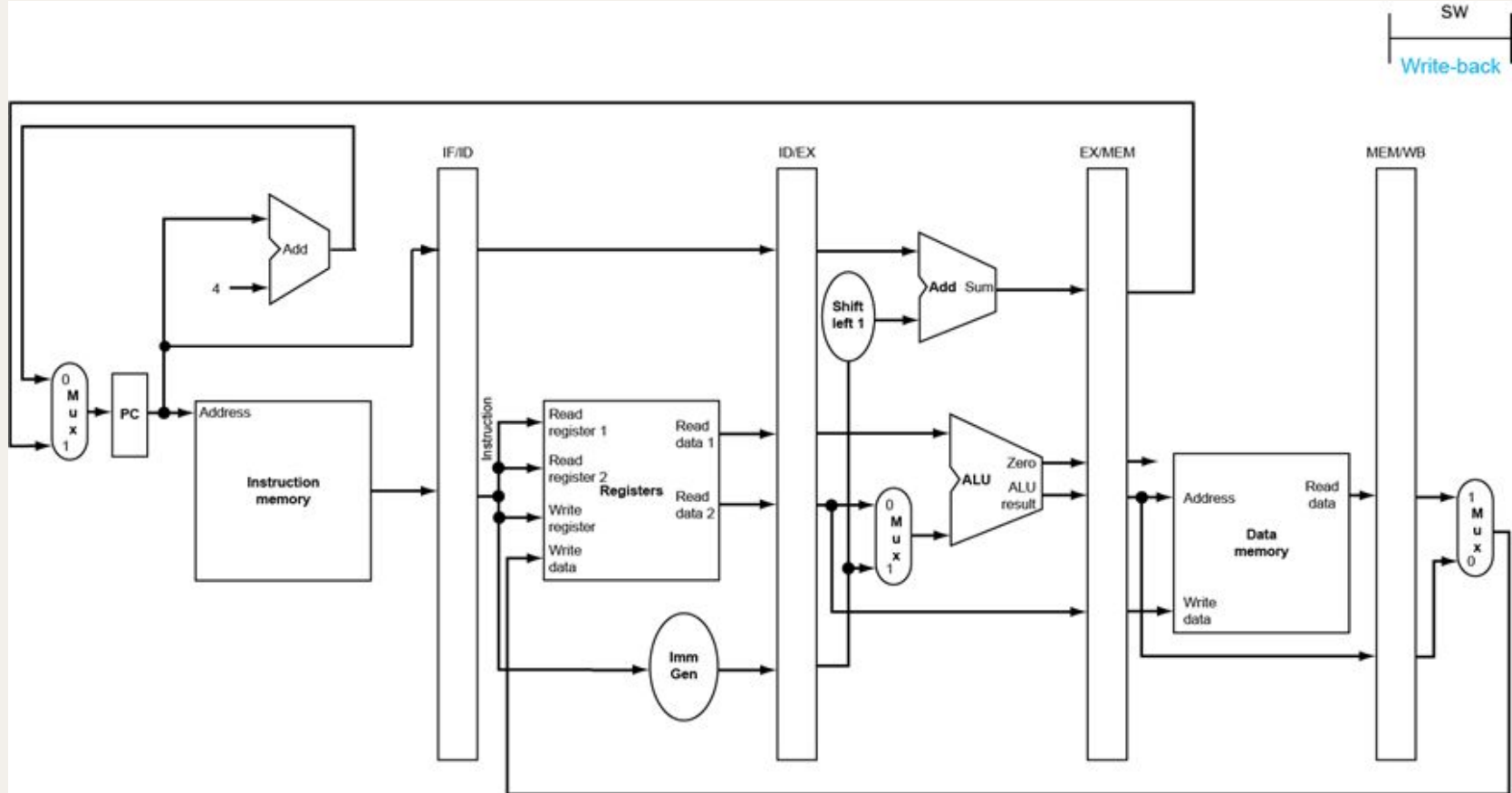
Andar EX para *store*



Andar MEM para instrução *store*

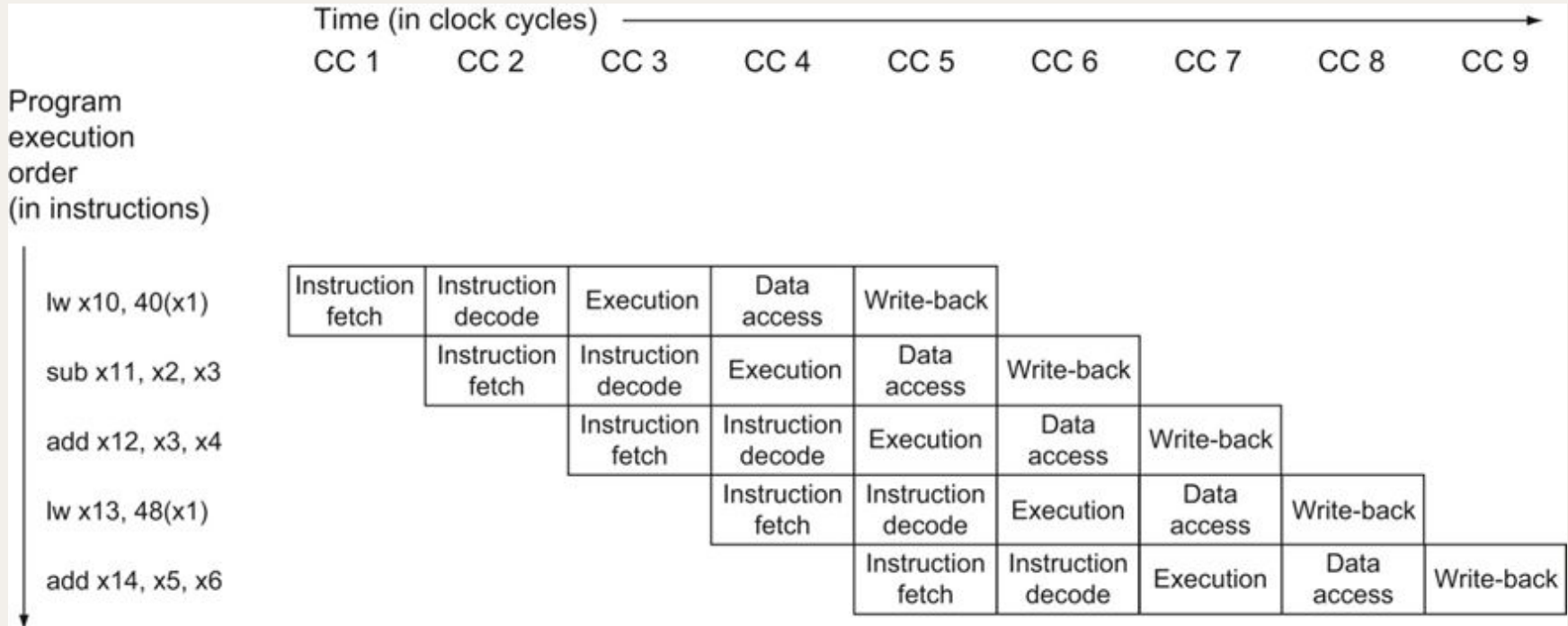


Andar WB para instrução *store*



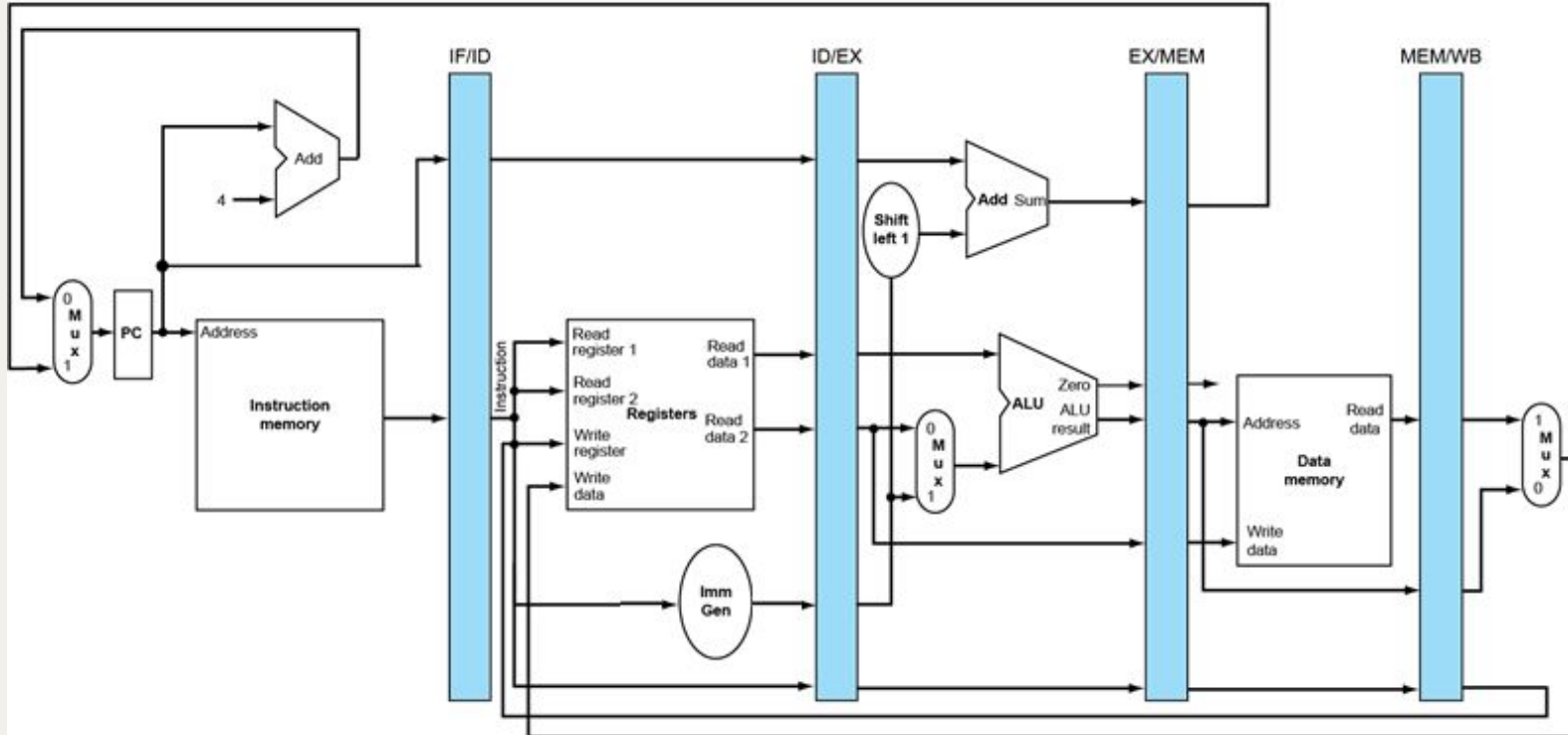
Estado de *pipeline* em cada ciclo

Esquema “tradicional”

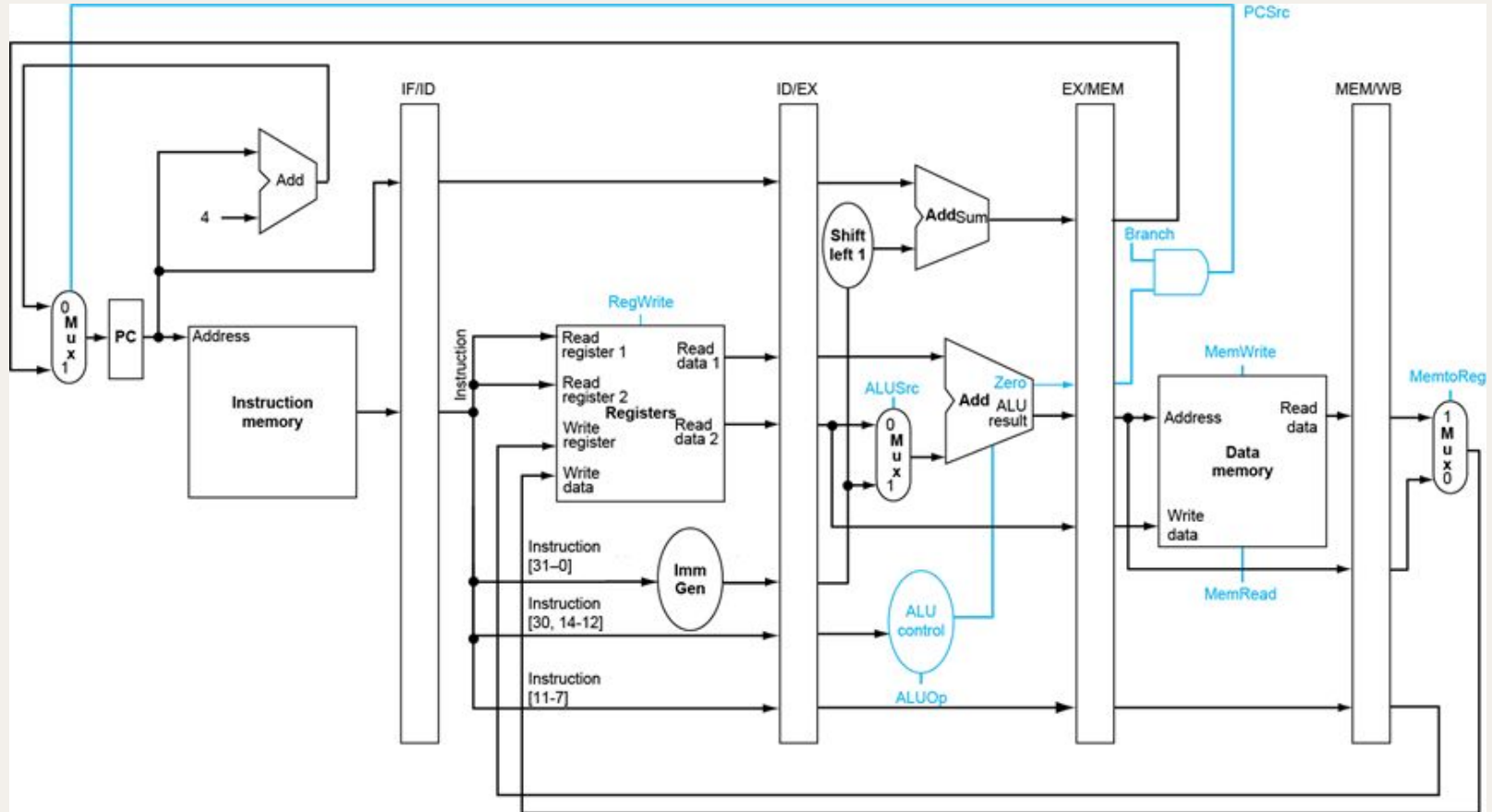


Estado de *pipeline* num ciclo específico (CC5)

add x14, x5, x6	lw x13, 48(x1)	add x12, x3, x4	sub x11, x2, x3	lw x10, 40(x1)
Instruction fetch	Instruction decode	Execution	Memory	Write-back

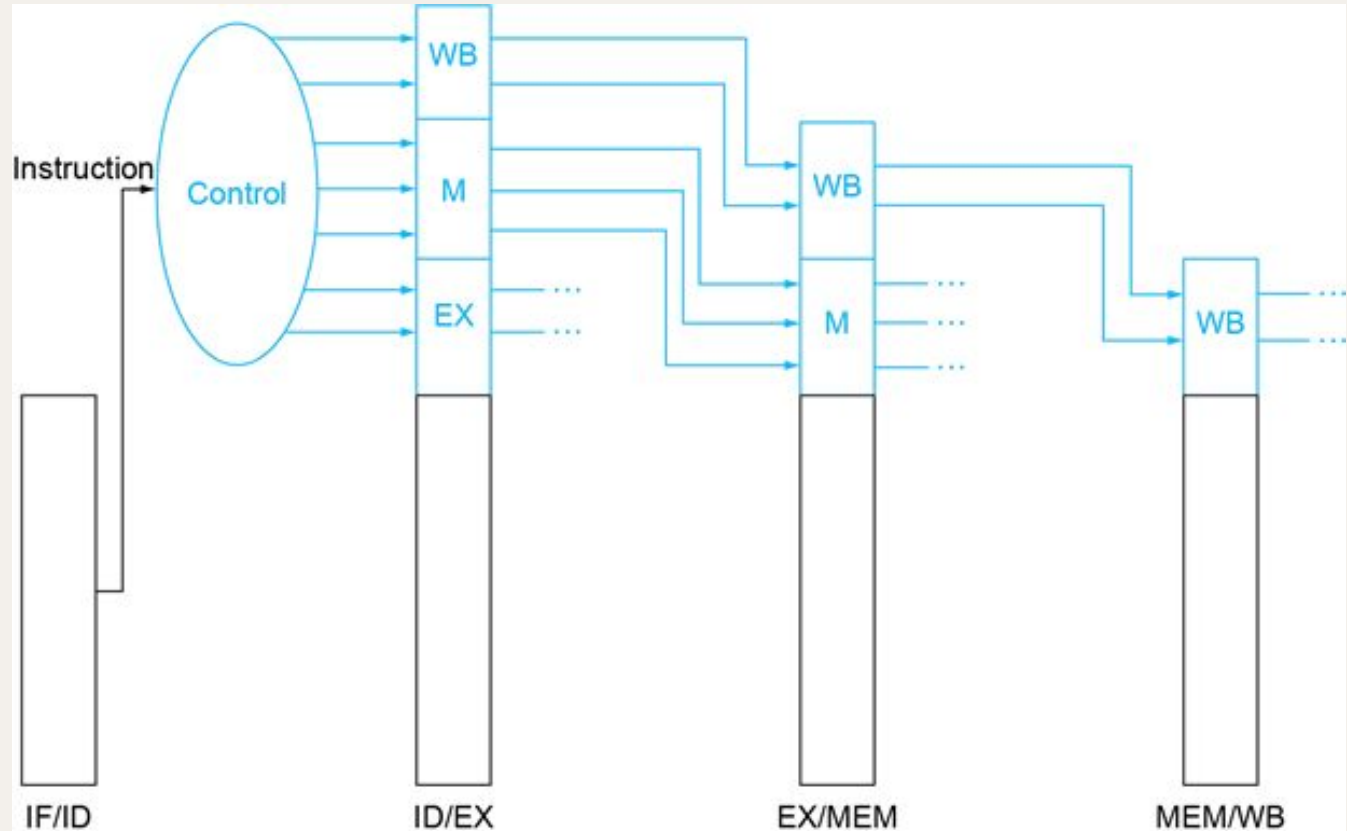


Sinais de controlo (simplificado)



Geração e propagação dos sinais de controlo

- O valor dos sinais de controlo é derivado da instrução em EX.
- Os valores são **propagados** até ao andar apropriado.
- Os sinais de controlo também são **“pipelined”**.



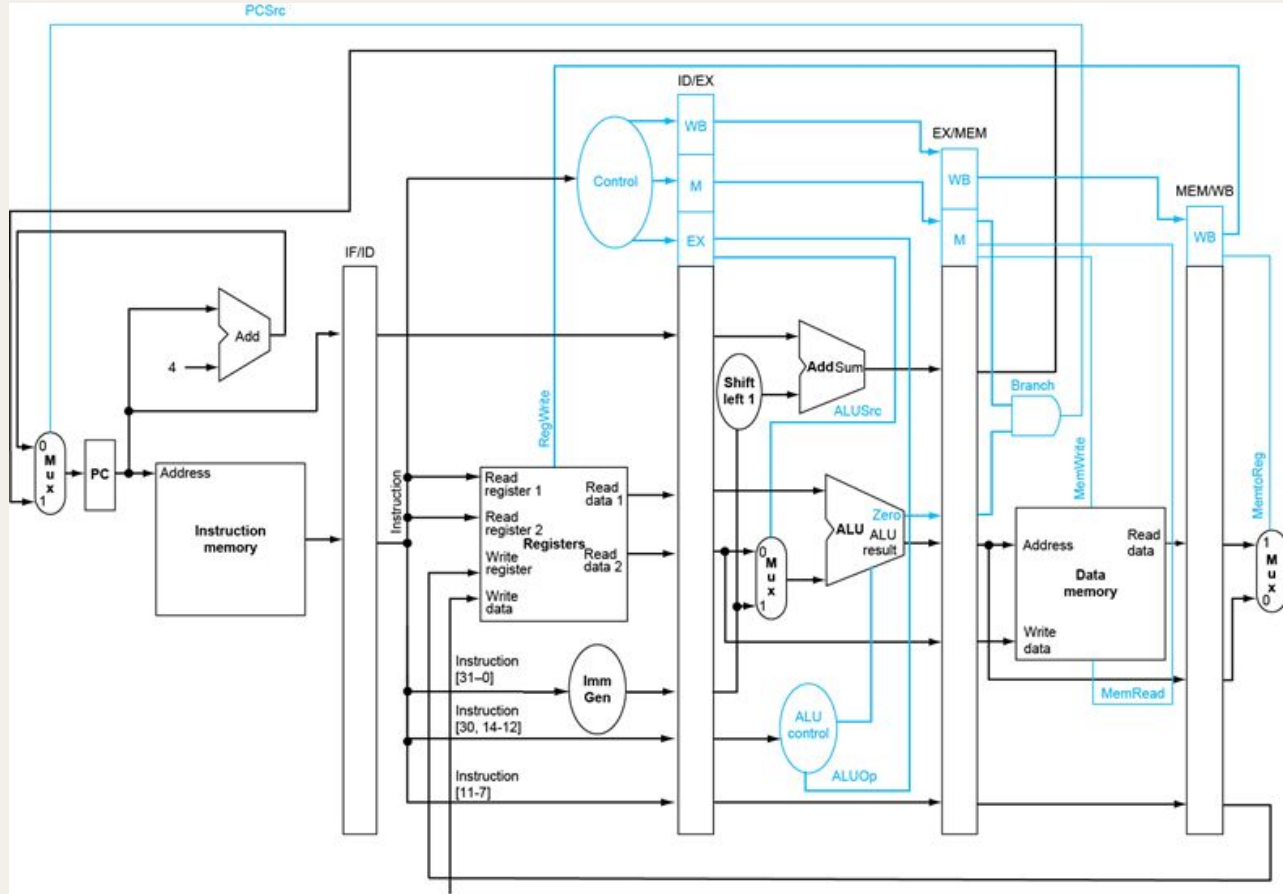
Especificação dos sinais de controlo

Como na versão uniciclo

Signal name	Effect when deasserted	Effect when asserted
RegWrite	None.	The register on the Write register input is written with the value on the Write data input.
ALUSrc	The second ALU operand comes from the second register file output (Read data 2).	The second ALU operand is the sign-extended, 12 bits of the instruction.
PCSrc	The PC is replaced by the output of the adder that computes the value of PC + 4.	The PC is replaced by the output of the adder that computes the branch target.
MemRead	None.	Data memory contents designated by the address input are put on the Read data output.
MemWrite	None.	Data memory contents designated by the address input are replaced by the value on the Write data input.
MemtoReg	The value fed to the register Write data input comes from the ALU.	The value fed to the register Write data input comes from the data memory.

Instruction	Execution/address calculation stage control lines		Memory access stage control lines			Write-back stage control lines	
	ALUOp	ALUSrc	Branch	Mem-Read	Mem-Write	Reg-Write	Memto-Reg
R-format	10	0	0	0	0	1	0
lw	00	1	0	1	0	1	1
sw	00	1	0	0	1	0	X
beq	01	0	1	0	0	0	X

Controlo completo de *pipeline*



Implementação II

Tratamento de conflitos de dados

Exemplo para estudar conflito de dados

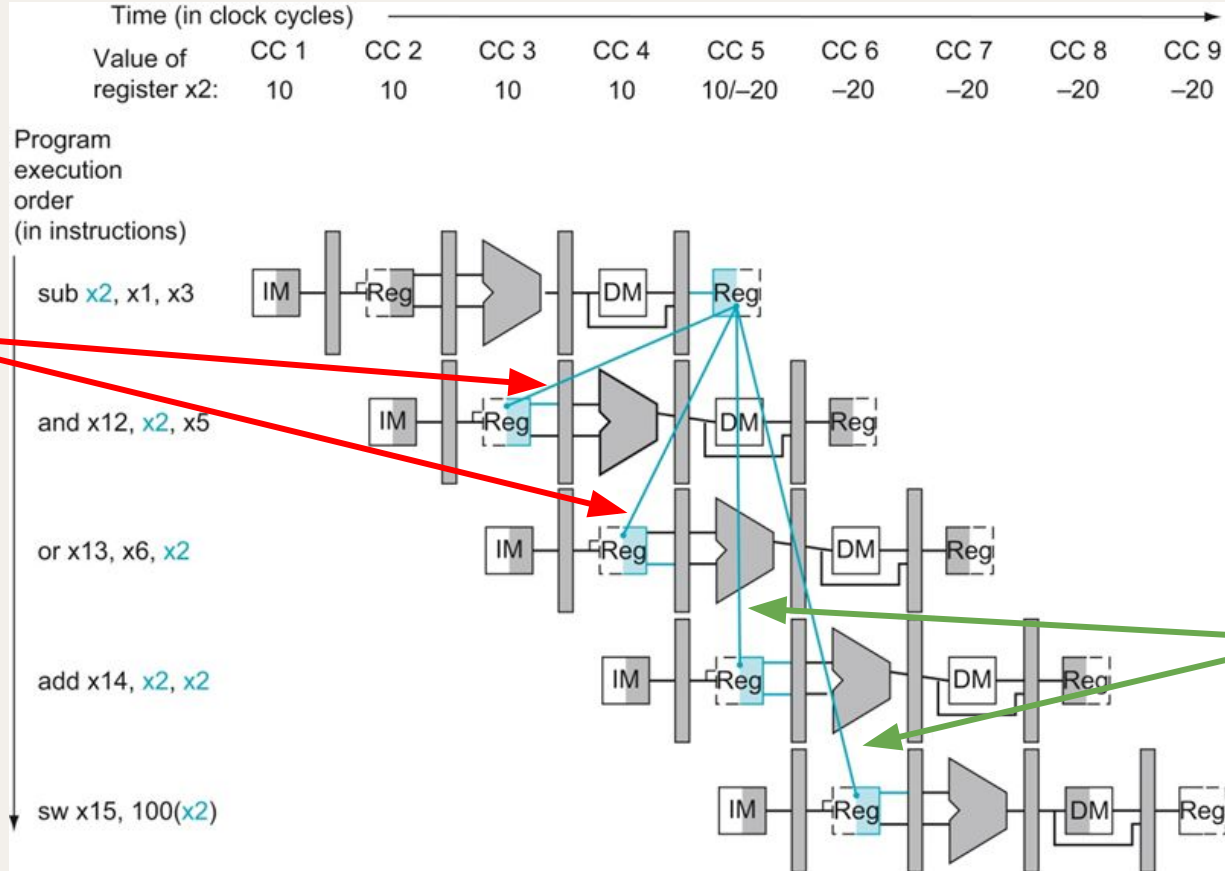
Considerar a sequência de instruções seguinte:

```
sub    x2,  x1, x3
and    x12, x2, x5
or     x13, x6, x2
add    x14, x2, x2
sw     x15, 100(x2)
```

Os conflitos deste exemplo podem ser evitados empregando atalhos.

Como detetar a necessidade de usar atalhos?

Dependências e conflitos



Conflitos

Sem conflitos

Atalhos

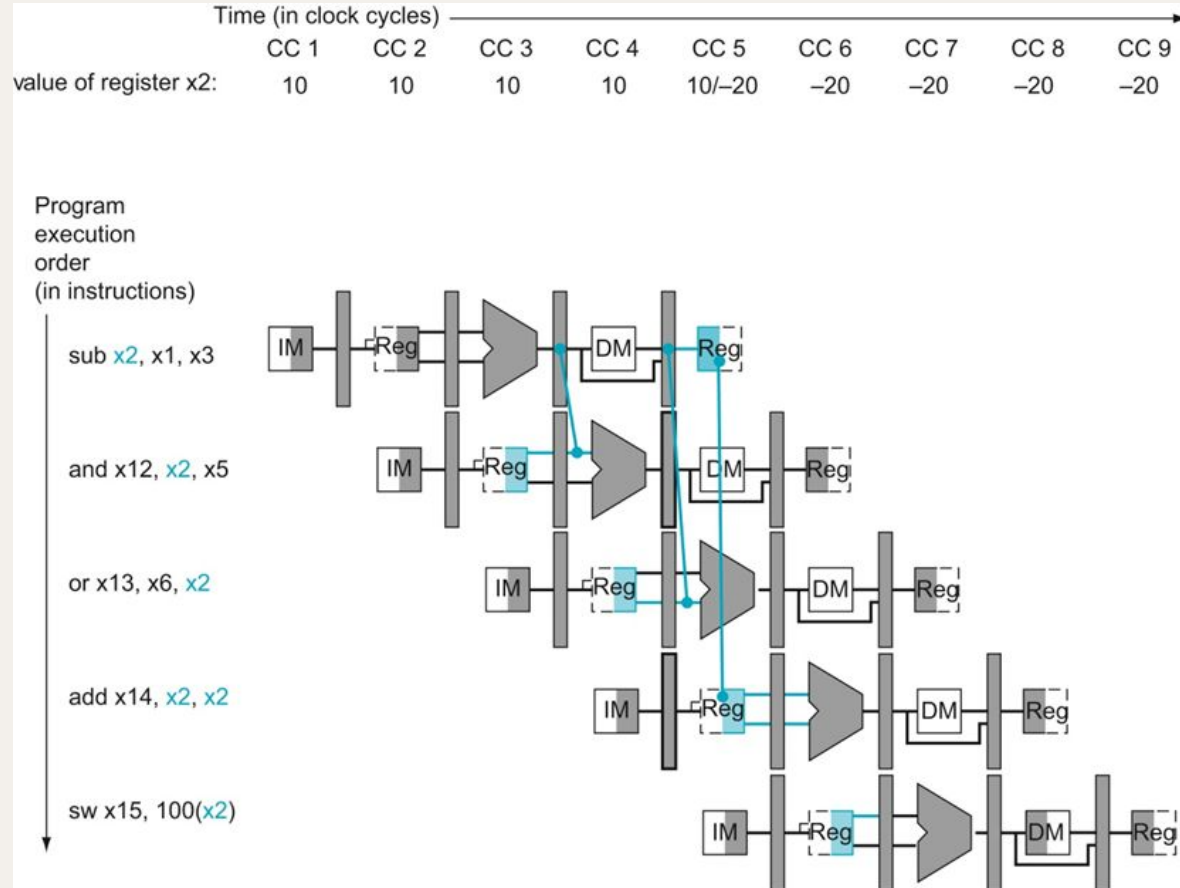
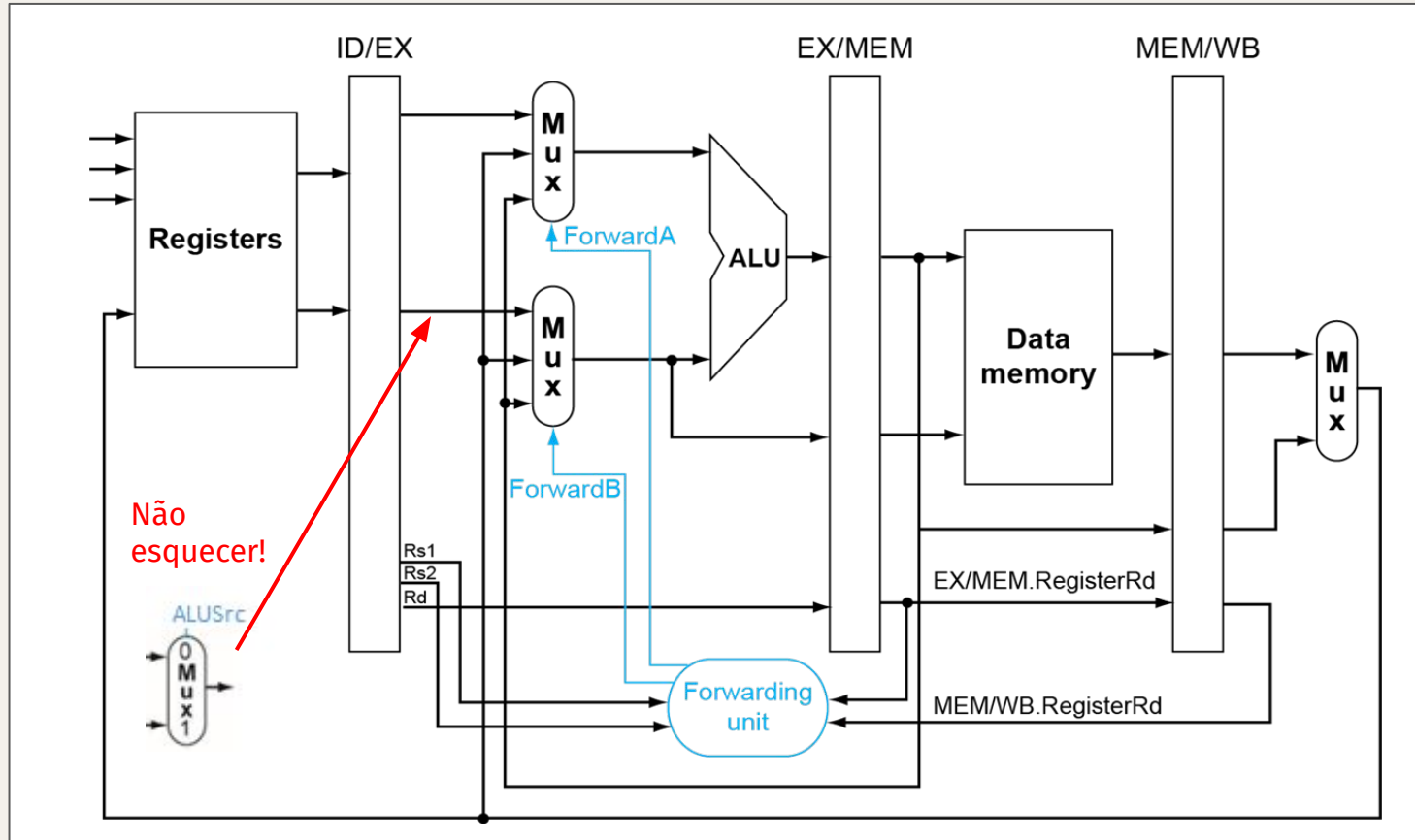
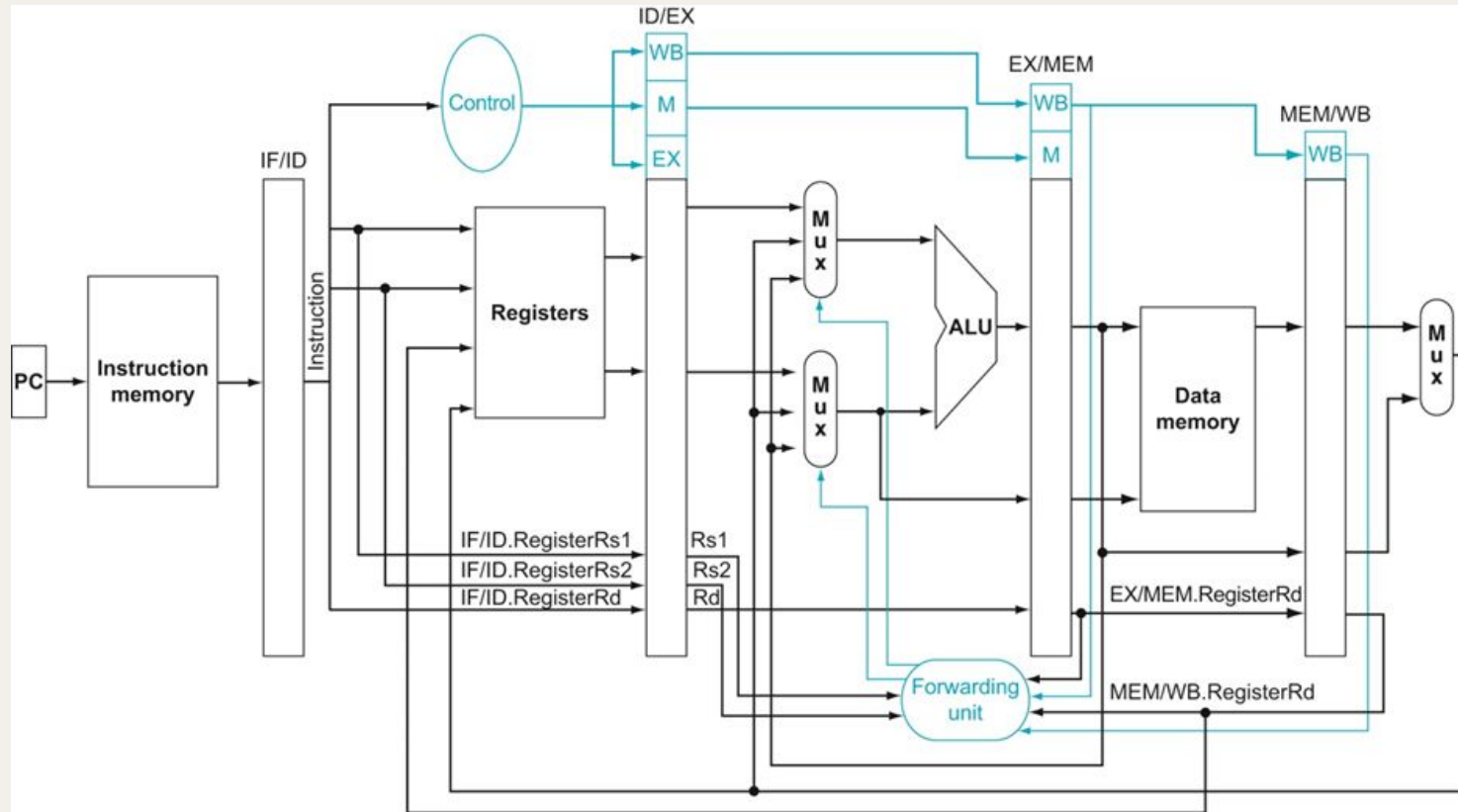


Diagrama com unidade de atalhos

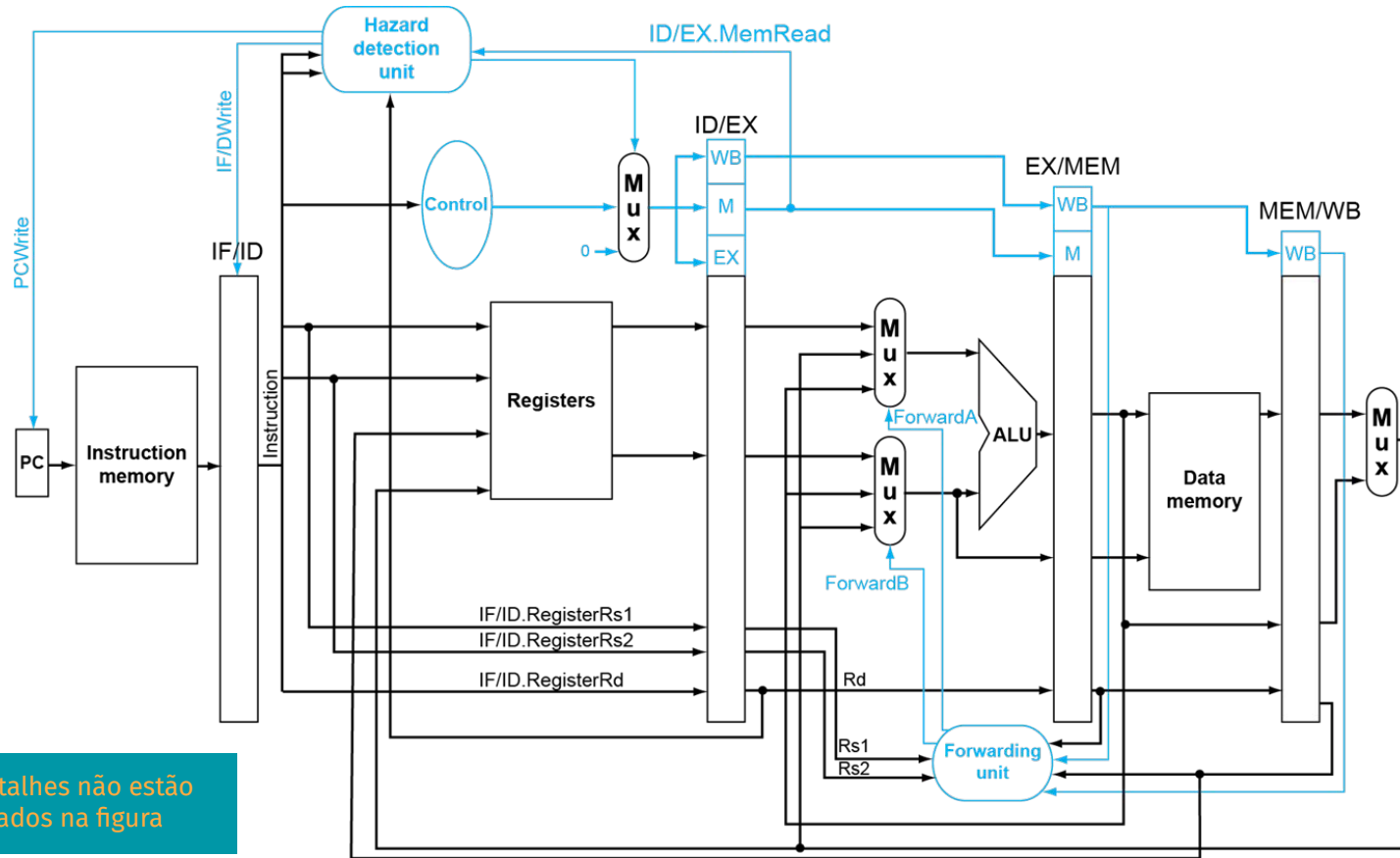


Percurso de dados *pipelined* com atalhos



Alguns detalhes não estão representados na figura (p.ex., *hardware* para tratamento de saltos)

Processador com tratamento de conflitos de dados completo



Alguns detalhes não estão representados na figura

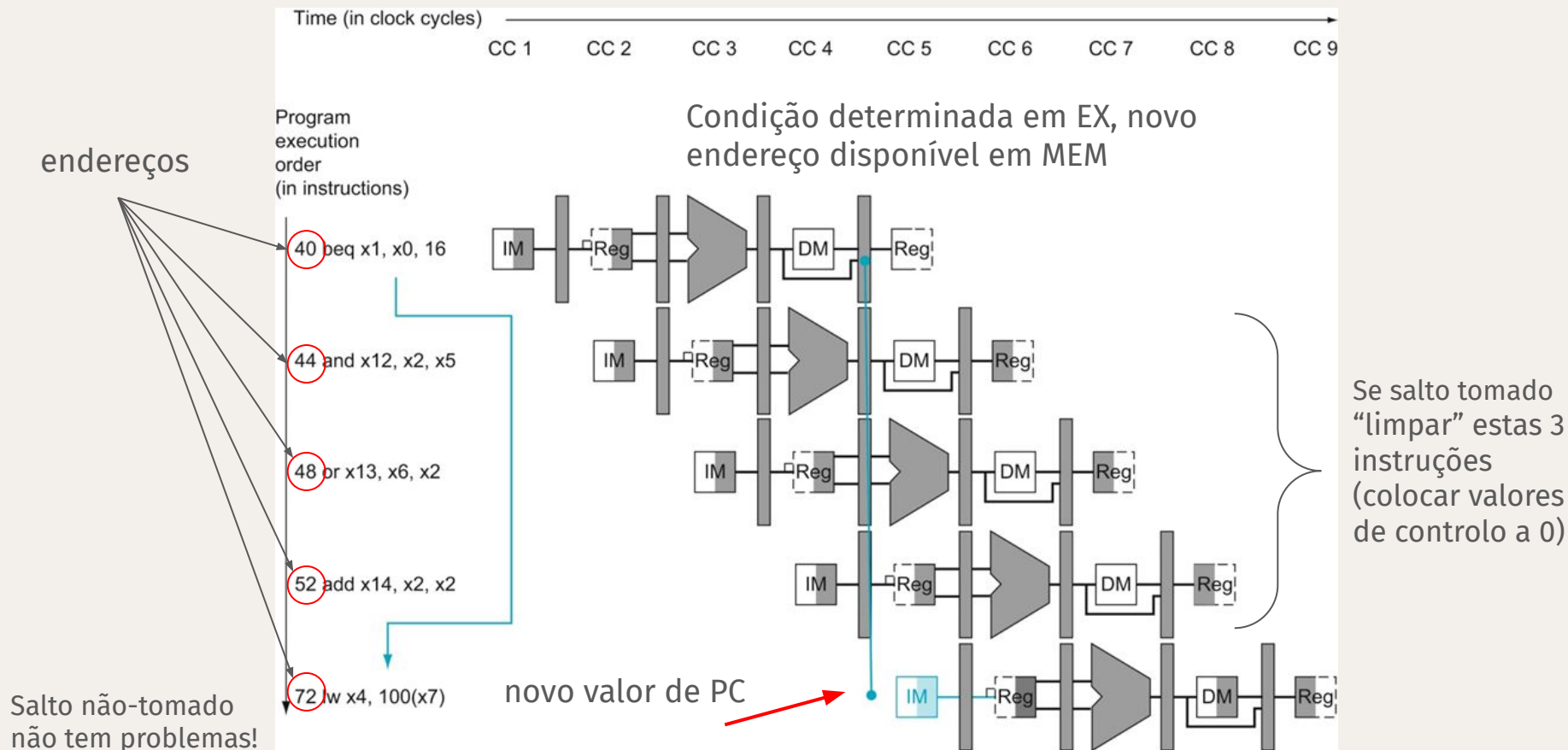
Implementação III

Tratamento de conflitos de controlo

Conflitos de controlo

- Instrução de salto condicional:
 - o valor de PC pode ser alterado para $PC + 4$ (salto não-tomado)
 - o valor de PC pode ser alterado para endereço da instrução de destino (salto **tomado**)
- Esquema 1 (mais simples):
 - anular instruções seguintes (nop) na *pipeline* até o destino ser conhecido
 - implica uma penalidade fixa para a instrução de salto condicional
 - na implementação de referência, são “perdidos” 3 ciclos
- Esquema 2:
 - assumir que o salto não é tomado (*predicted-not-taken*)
 - a *pipeline* opera como se o salto condicional fosse uma instrução qualquer
 - quando a condição é determinada como verdadeira, anular as instruções

Conflito de controlo: salto tomado (esquema 2)



Impacto dos saltos condicionais (em média)

- Aumento de desempenho em comparação com CPU sem *pipeline* (assumir CPI=1)

$$\text{Speedup de pipeline} = \frac{\text{n.º de andares}}{1 + \text{ciclos de protelamento causados por saltos}}$$

- Os ciclos de protelamento por instrução (em média) devido a saltos dependem de:
 - taxa de ocorrência (frequência) média das instruções de salto (B)
 - penalidade do salto (P)

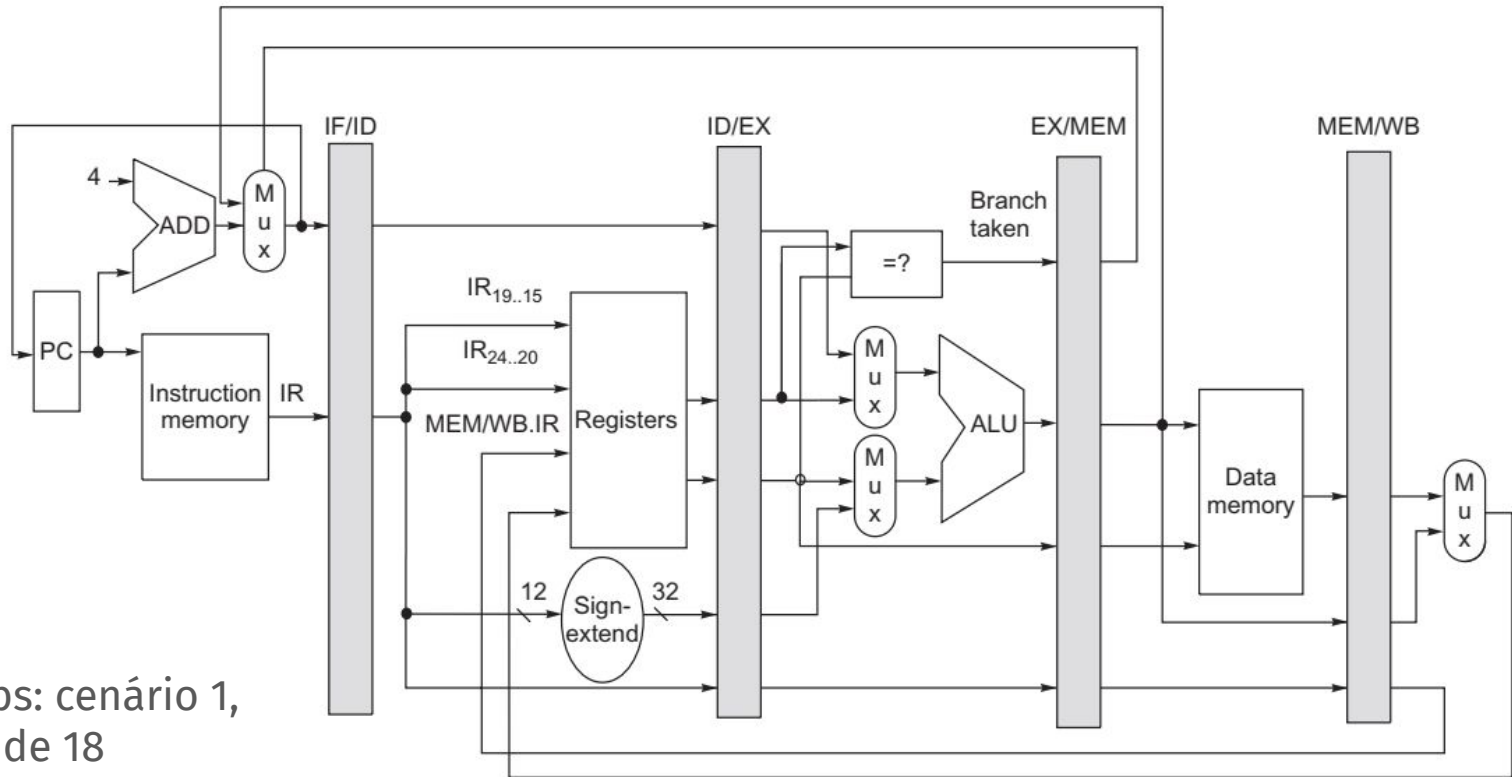
$$\text{Speedup de pipeline} = \frac{\text{n.º de andares}}{1 + B \times P}$$

Incremento de CPI devido a saltos

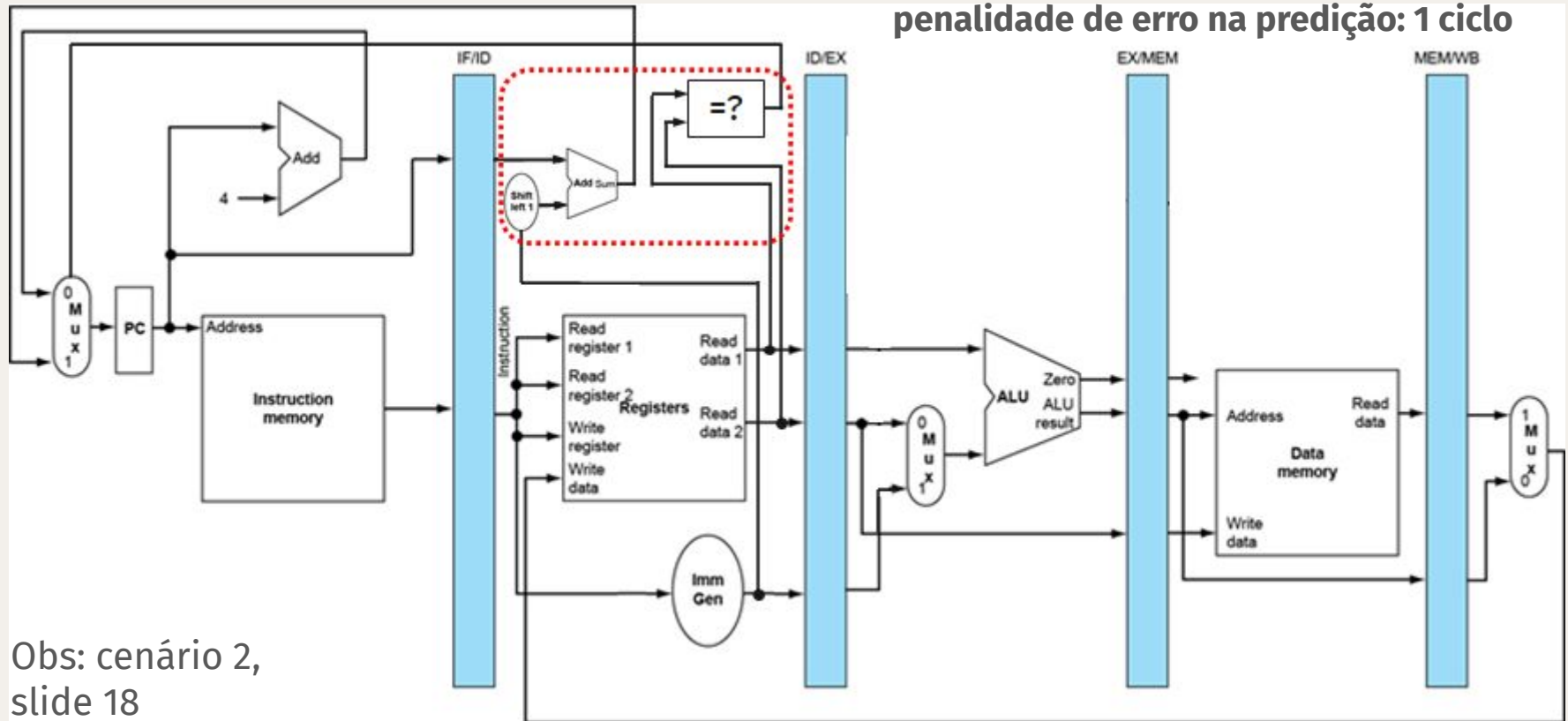
Para a implementação de *pipeline* estudada:

		Evento		
		Não-tomado	Tomado	Total
Frequência		6 %	10 %	16 %
Esquema				
Fixo (1)		0,18	0,3	0,48
Predição: não tomado (2)		0	0,3	0,3

Reduzir impacto de erros de previsão: original



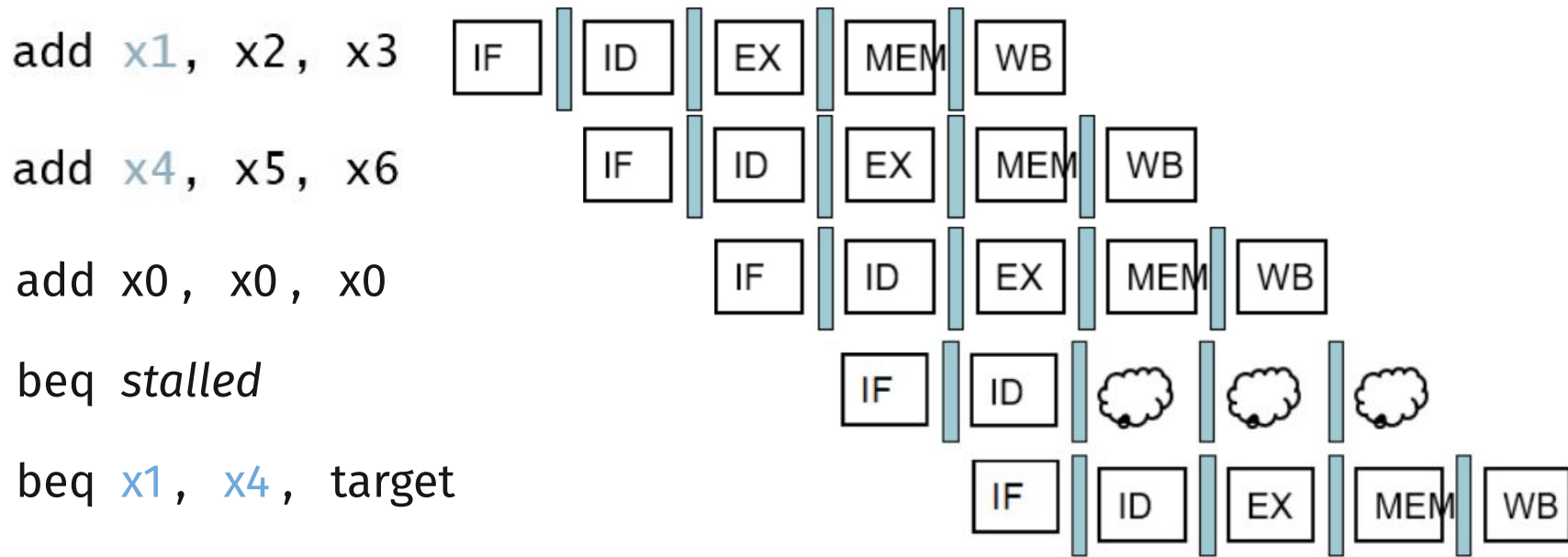
Reduzir impacto de erros de predição: decisão de salto na etapa ID



Conflitos de dados em saltos condicionais (I)

Existe um conflito se um dos registos de comparação é o destino da 2.^a ou 3.^a instrução precedente.

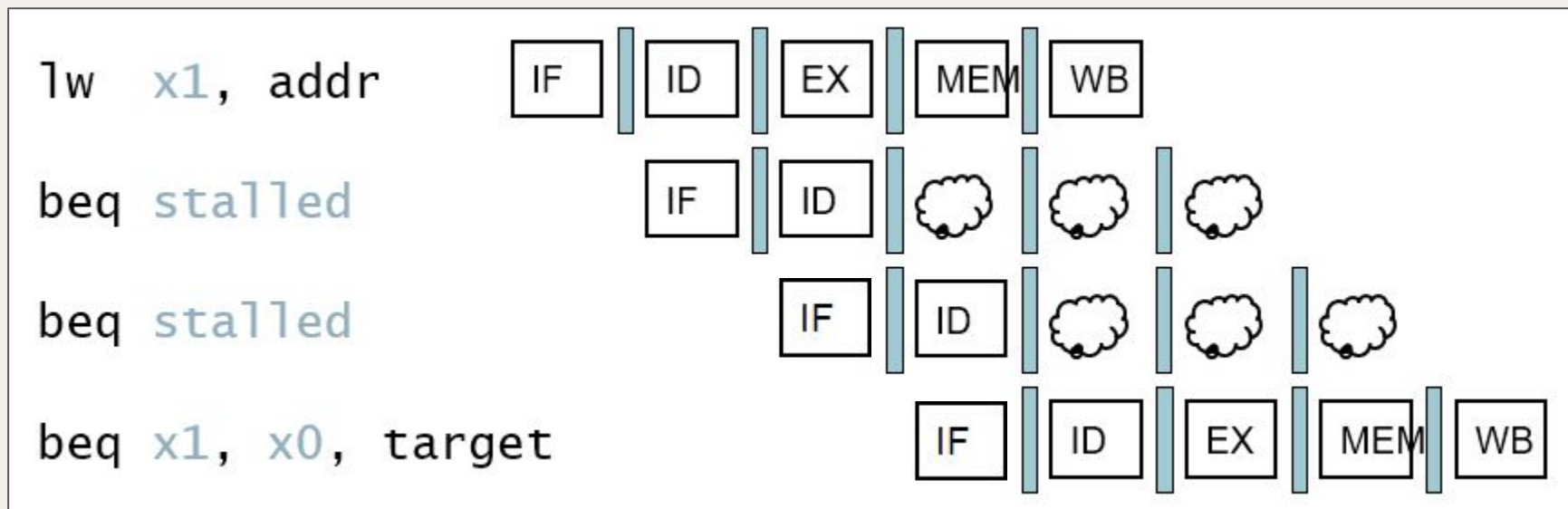
É necessário introduzir um ciclo de protelamento.



Conflitos de dados em saltos condicionais (II)

Se um registo usado na comparação é o destino de uma instrução de *load* executada imediatamente antes:

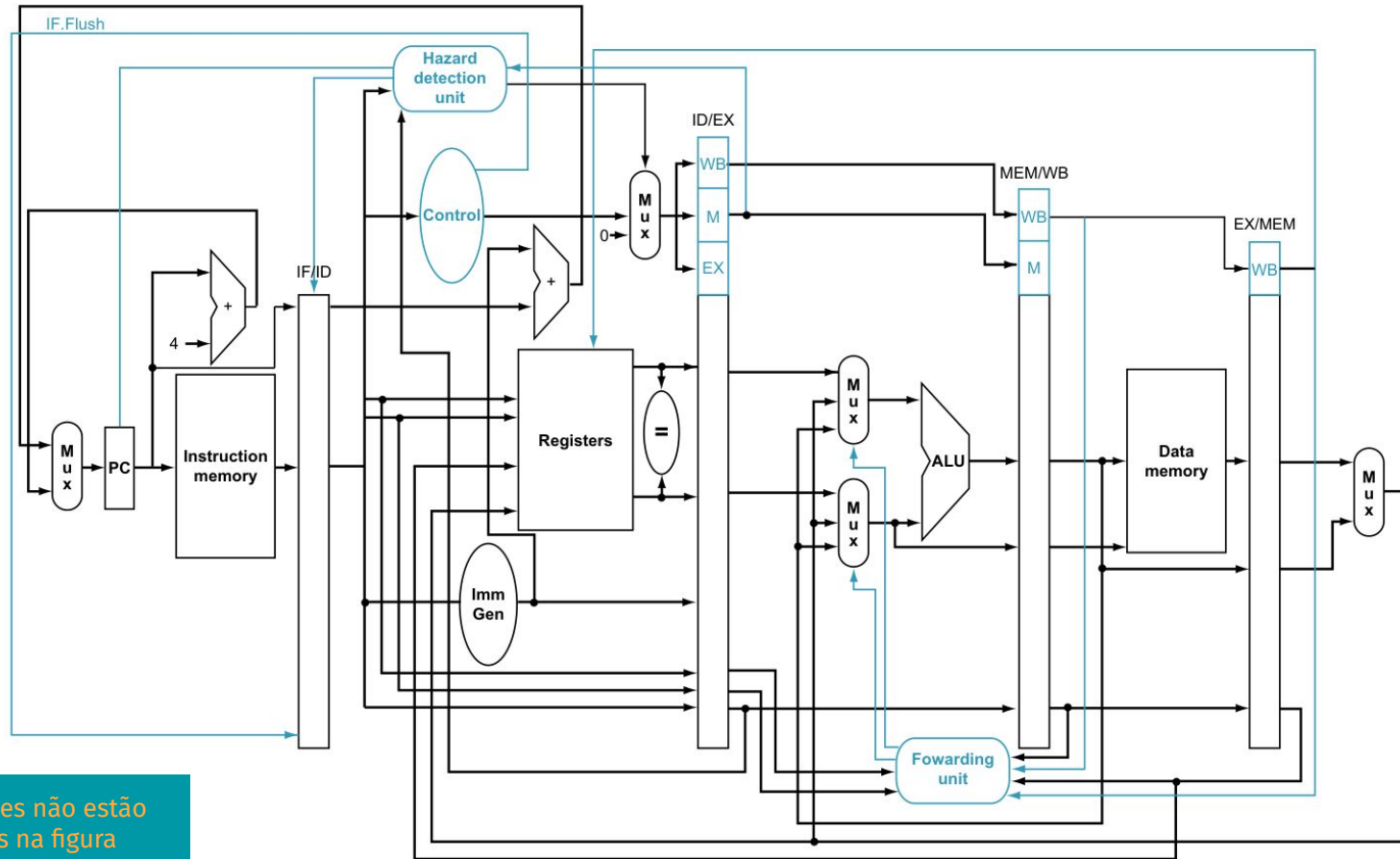
É necessário introduzir dois ciclos de protelamento.



Decisão de saltos condicionais na etapa ID

- A tomada de decisão de salto ocorre, na versão de base, na etapa MEM.
- A tomada de decisão de salto na etapa ID é vantajosa, ao permitir conhecer a decisão 2 ciclos mais cedo, se não ocorrer conflito entre instrução de salto e instruções anteriores.
- Decisão de salto em ID não beneficia dos atalhos estudados ALU-ALU e MEM-ALU (ID precede sempre EX).
- Os dois exemplos anteriores mostram que os conflitos anulam a poupança de 2 ciclos resultante da antecipação da decisão de salto de MEM para ID.

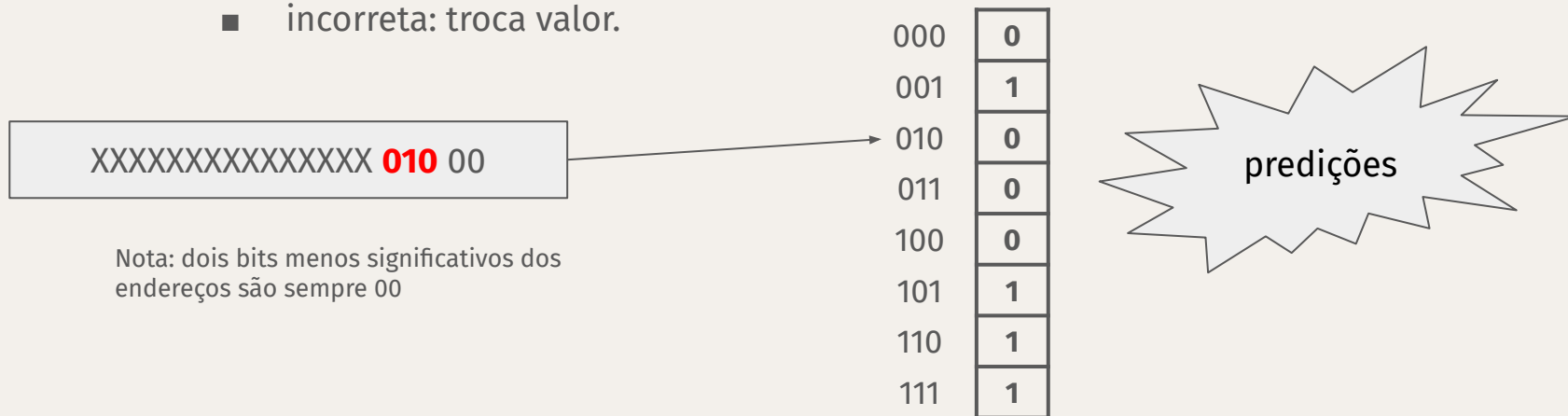
Versão final



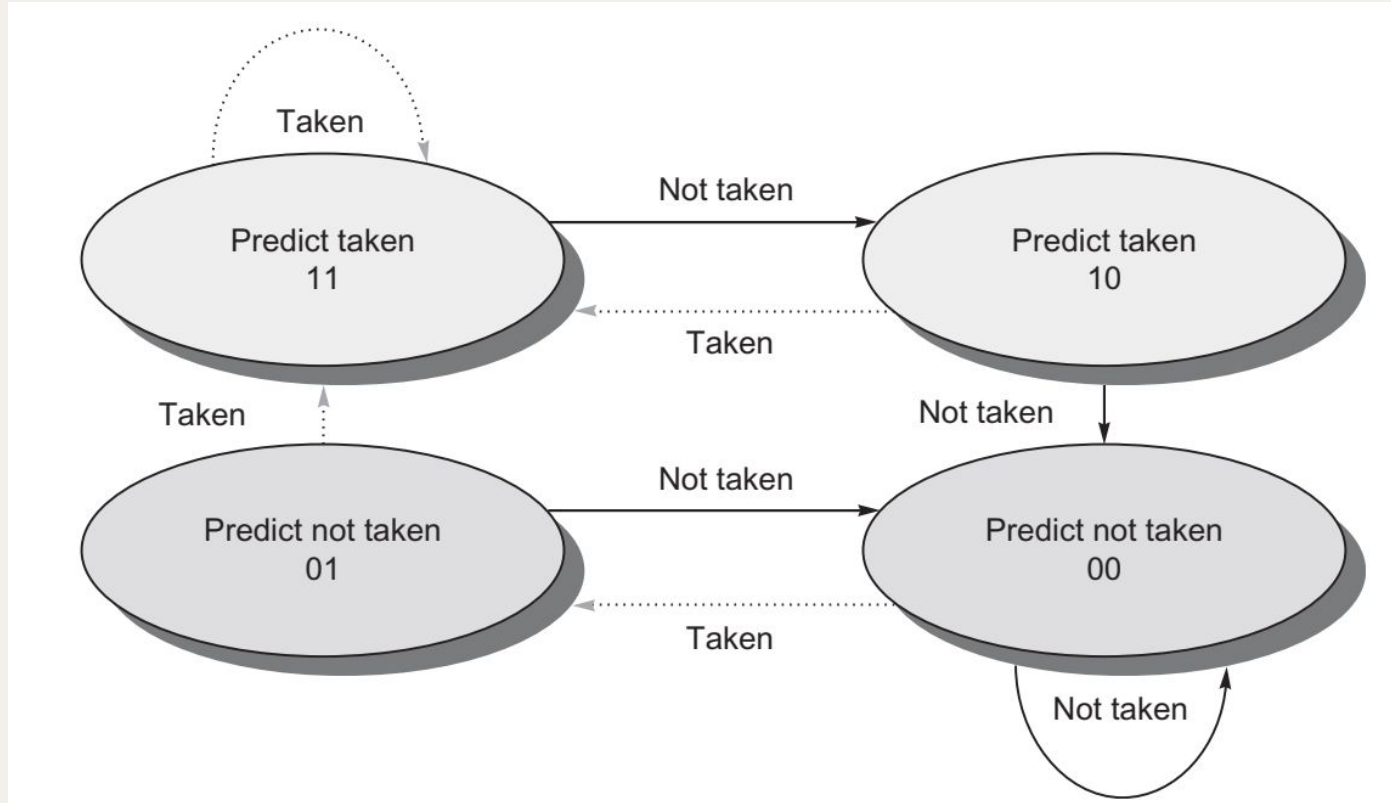
Alguns detalhes não estão representados na figura

Melhorar a predição de saltos

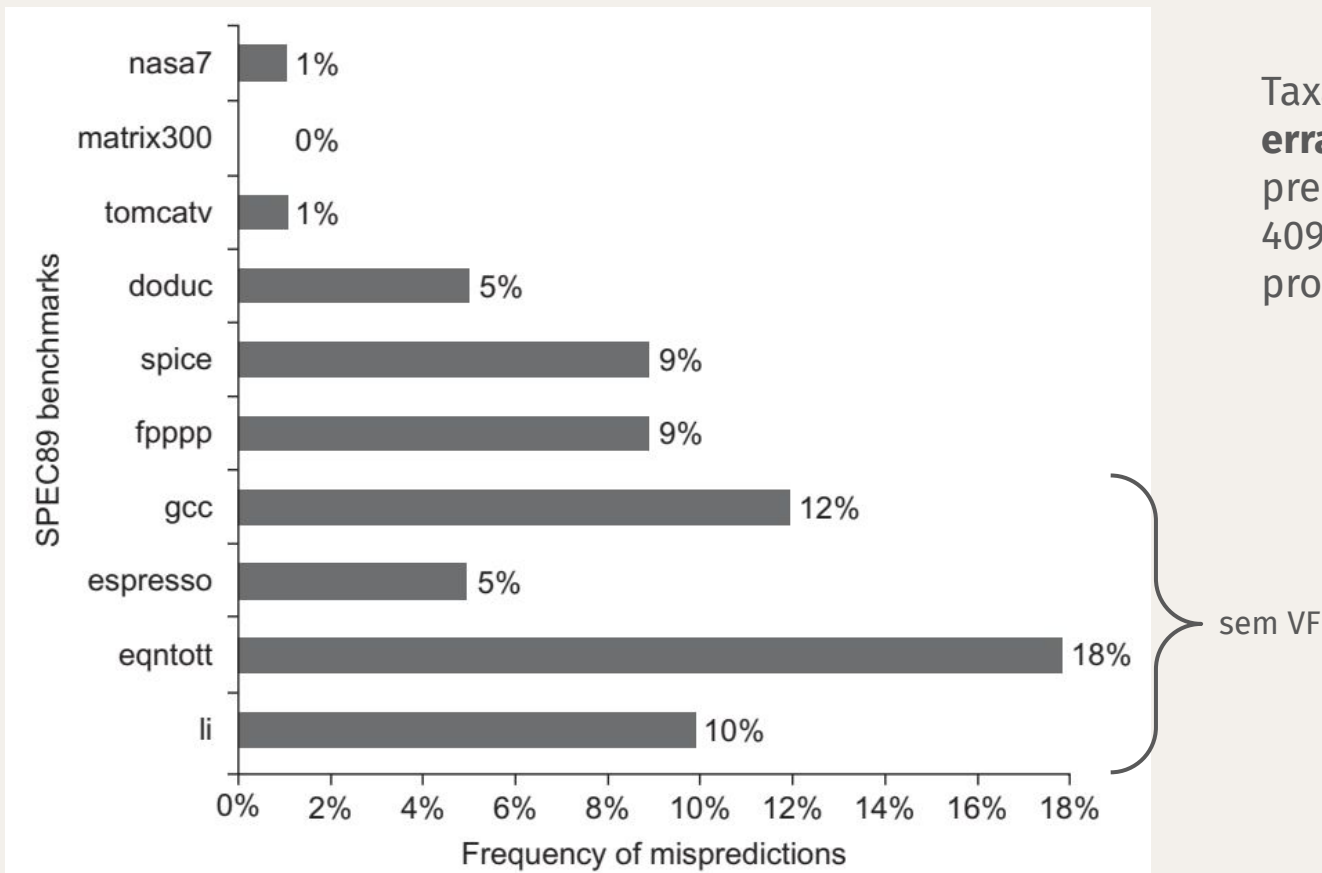
- Objetivo: melhorar a predição de saltos usando informação “dinâmica”
 - guardar informação sobre o comportamento anterior do programa
- Técnica mais simples: *branch-prediction buffer* (ou *branch history table*)
 - Memória indexada pela parte menos significativa do *endereço da instrução*
 - Usa um bit para registrar se o salto foi tomado (da última vez que foi executado)
 - Quando a predição é usada:
 - correta: mantém valor;
 - incorreta: troca valor.



Predição dinâmica com 2 bits por endereço



Resultados empíricos



Taxa de predições **erradas** para um preditor de 2 bits com 4096 entradas para os programas SPEC89

Conclusão

- Aplicar “pipelining” aumenta a eficiência
 - A ideia básica é simples;
 - Os detalhes são (muito) complicados (por exemplo, detecção de conflitos).
- “Pipelining” aumenta o débito (nº de instruções completadas por segundo)
 - A latência de cada instrução não é reduzida.
- A execução concorrente de instruções faz com que as dependências entre instruções tenham impacto no desempenho.
 - A organização interna do CPU determina a dimensão desse impacto.
- Atalhos podem evitar muitos protelamentos.
- Saltos condicionais podem ter um impacto grande no desempenho.

Informação detalhada

- Secções 4.6 - 4.9 de David A. Patterson & John L. Hennessy
“Computer Organization and Design”, RISC-V edition, 2nd ed., Elsevier.